

数据结构与算法

图

图

- 7.1 图的定义和术语
- 7.2 图的存储表示
- 7.3 图的遍历
- 7.4 图的最小代价生成树
- 7.5 有向无环图及应用
- 7.6 最短路径

7.1 图的定义和术语

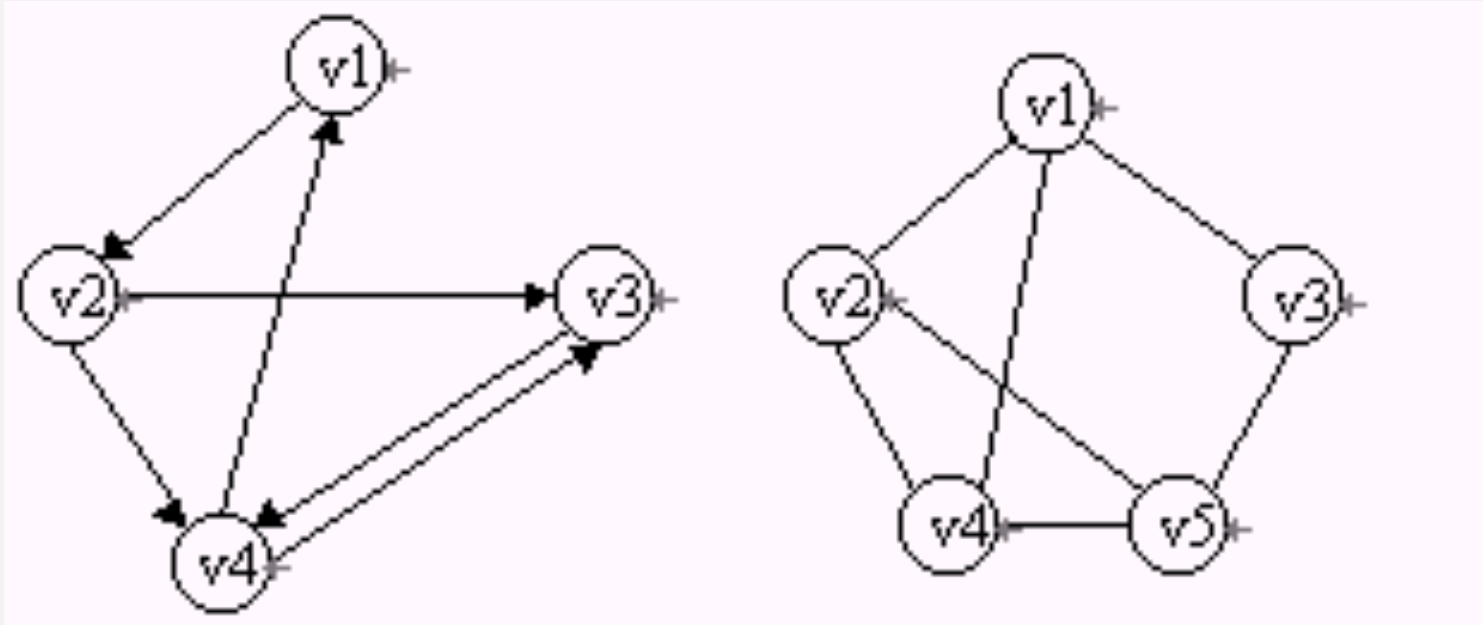
- 图（Graph）是一种较线性表和树更为复杂的非线性结构。
- 在线性结构中，结点之间的关系是线性关系，除开始结点和终端结点外，每个结点只有一个直接前趋和直接后继。
- 在树形结构中，结点之间的关系实质上是层次关系，同层上的每个结点可以和下一层的零个或多个结点（即孩子）相关，但只能和上一层的一个结点（即双亲）相关（根结点除外）。

7.1 图的定义和术语

- 然而在图结构中，对结点（图中常称为顶点）的前趋和后继个数都是不加限制的，即结点之间的关系是任意的。
- 图中任意两个结点之间都可能相关。
- 由此，图的应用极为广泛，特别是近年来的迅速发展，已渗透到诸如语言学、逻辑学、物理、化学、电讯工程、计算机科学以及数学的其它分支中。

7.1 图的定义和术语

- 图是由结点的有限集合 V 和边的集合 E 组成。其中，为了与树形结构加以区别，在图结构中常常将结点称为顶点，若两个顶点之间存在一条边，就表示这两个顶点具有相邻关系。



7.1 图的定义和术语

图(Graph)是一种网状数据结构，由顶点和边两个有限集合组成。其形式化定义如下：

$$\text{Graph} = (V, R)$$

$$V = \{x \mid x \in \text{DataObject}\}$$

$$R = \{VR\}$$

$$VR = \{ \langle x, y \rangle \mid P(x, y) \wedge (x, y \in V) \}$$

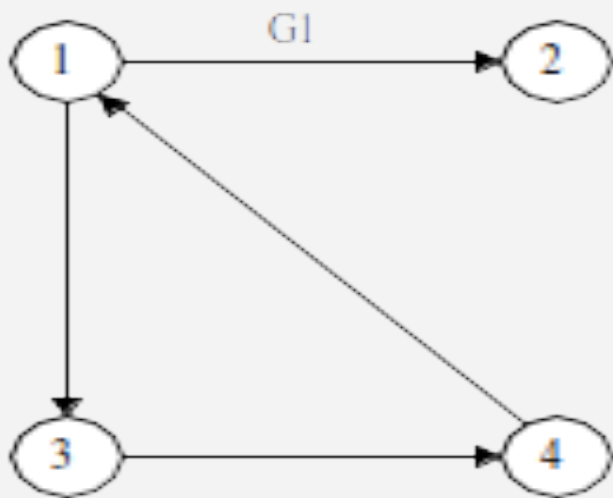
DataObject为一个集合，该集合中的所有元素具有相同的特性。V中的数据元素通常称为顶点(vertex)，VR是两个顶点之间关系的集合。P(x, y)表示x和y之间有特定的关联属性P。

7.1 图的定义和术语

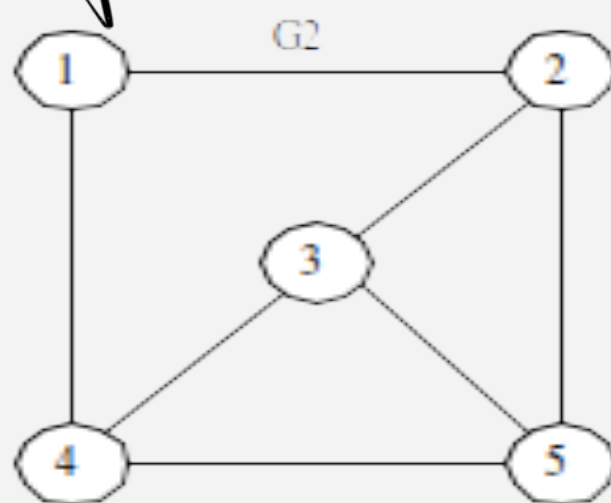
- 顶点：图中的数据元素通常称为顶点。 V 是顶点的有穷非空集合。
- VR 是两个顶点之间关系的集合。
- 若 $\langle x, y \rangle \in VR$ ，则 $\langle x, y \rangle$ 表示从顶点 x 到顶点 y 的一条弧（arc）。并称 x 为弧尾（tail）或起点，称 y 为弧头（head）或终点，此时图中的边是有方向的。如果“边”是有方向的，称由顶点集和边集构成的图为有向图。

7.1 图的定义和术语

- 若 $\langle x, y \rangle \in VR$ ，必有 $\langle y, x \rangle \in VR$ ，即VR是对称关系，这时以无序对 (x, y) 来代替两个有序对，表示x和y之间的一条边（edge），称顶点x和顶点y之间存在一条无向边，此时的图称为无向图。



(a) $G1$



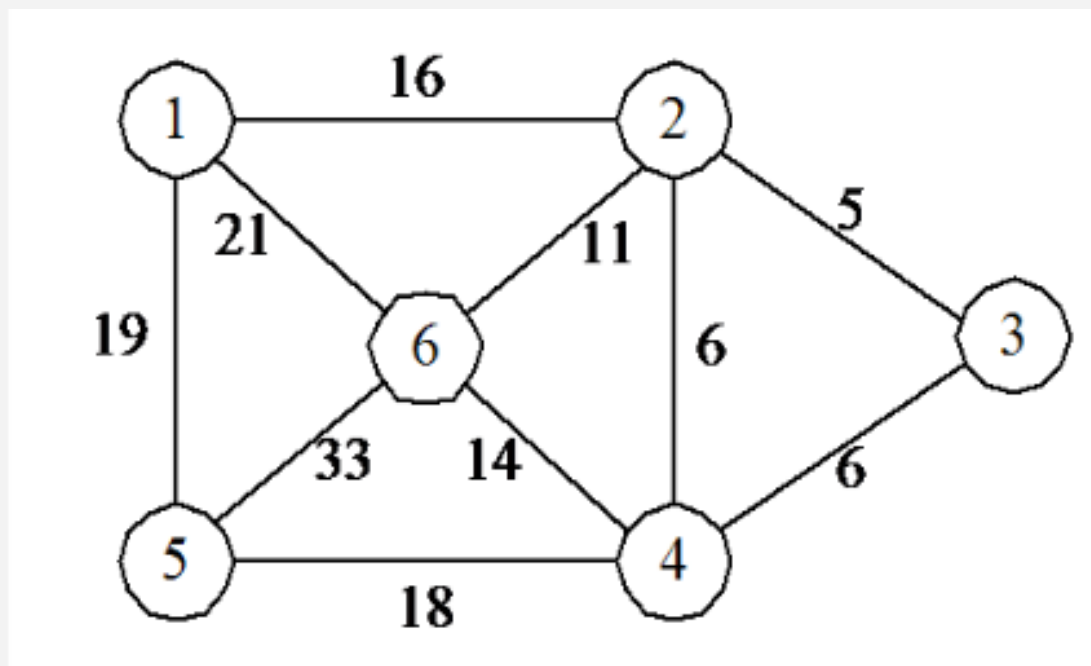
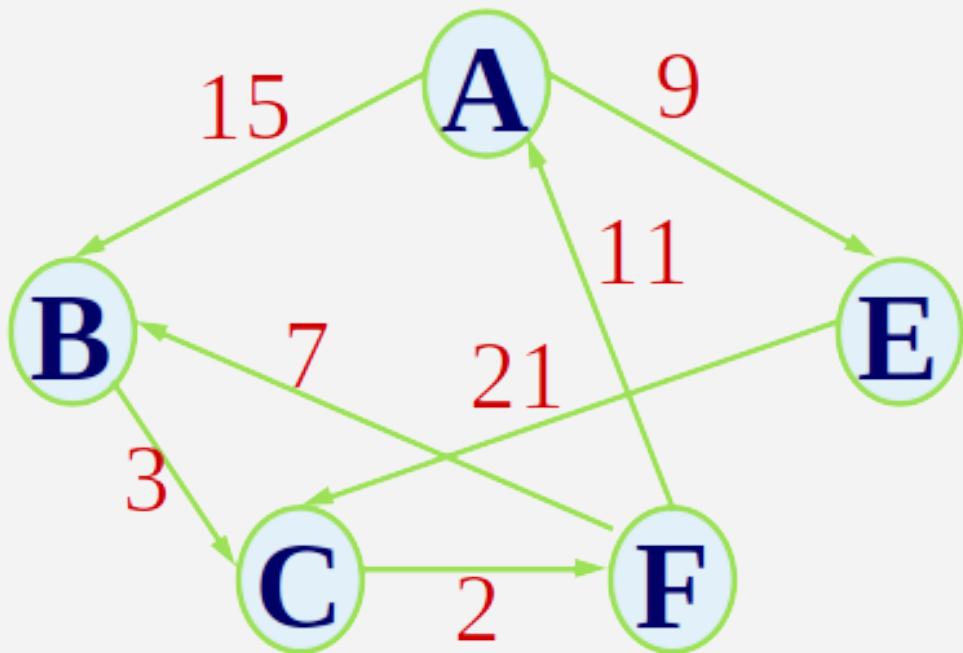
(b) $G2$

7.1 图的定义和术语

- 在实际应用中，有时图的边或弧上往往与具有一定意义的数有关，即每一条边都有与它相关的数，称为权，这些权可以表示从一个顶点到另一个顶点的距离或耗费等信息。
- 将这种带权的图叫做网。

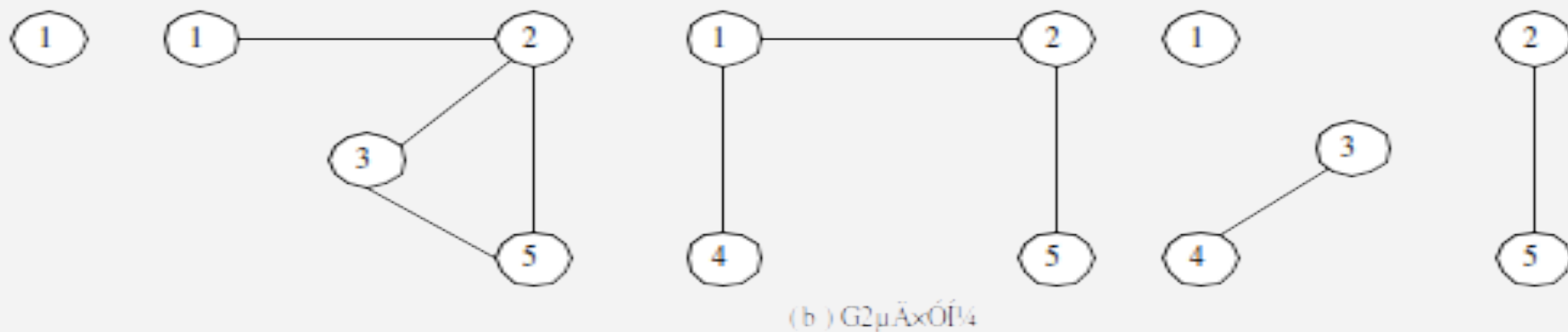
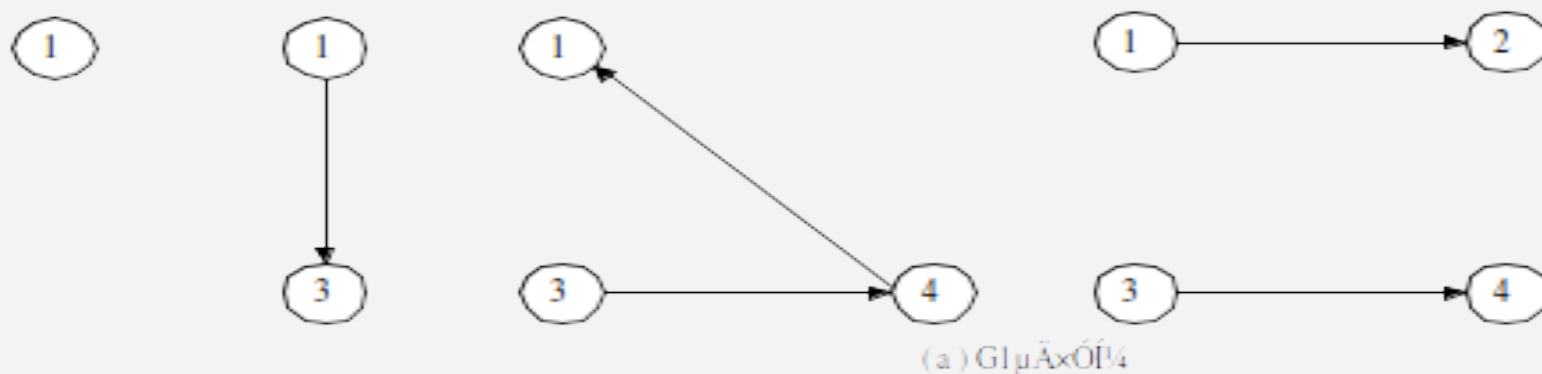
7.1 图的定义和术语

- 边带权的有向图称作有向网，边带权的无向图称为无向网。



7.1 图的定义和术语

- 设图 $G=(V, \{E\})$ 和图 $G_\phi=(V_\phi, \{E_\phi\})$, 且 $V_\phi \subseteq V$, $E_\phi \subseteq E$, 则称 G_ϕ 为 G 的子图。



7.1 图的定义和术语

- 设 n 表示图中顶点的个数，用 e 表示图中边或弧的数目，并且不考虑图中每个顶点到其自身的边或弧。即若 $\langle v_i, v_j \rangle \in VR$ ，则 $v_i \neq v_j$ 。对于无向图而言，其边数 e 的取值范围是 $0 \sim n(n-1)/2$ 。
$$C_n^2 = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$
- 称有 $n(n-1)/2$ 条边的无向图为完全图。
- 称有 $n(n-1)$ 条边的有向图为有向完全图。
$$A_n^2 = n(n-1)$$
- 对于有很少条边的图（ $e < n \log n$ ）称为稀疏图，反之有较多条边的图称为稠密图。

7.1 图的定义和术语

- 对于无向图 $G=(V, \{E\})$ ，如果边 $(v, v') \in E$ ，则称顶点 v, v' 互为邻接点，即 v, v' 相邻接。边 (v, v') 依附于顶点 v 和 v' ，或者说边 (v, v') 与顶点 v 和 v' 相关联。对于有向图 $G=(V, \{A\})$ 而言，若边 $\langle v, v' \rangle \in A$ ，则称顶点 v 邻接到顶点 v' ，顶点 v' 邻接自顶点 v ，或者说边 $\langle v, v' \rangle$ 与顶点 v 和 v' 相关联。

7.1 图的定义和术语

- 对于无向图而言，顶点 v 的度是指和 v 相关联边的数目，记作 $TD(v)$ 。例如：图 G_2 中顶点 v_3 的度是3， v_1 的度是2；
- 在有向图中顶点 v 的度有出度和入度两部分，其中以顶点 v 为终点的边的数目成为该顶点的入度，记作 $ID(v)$ ，以顶点 v 为起点的边的数目称为该顶点的出度，记作 $OD(v)$ ，则顶点 v 的度为 $TD(v) = ID(v) + OD(v)$ 。

7.1 图的定义和术语

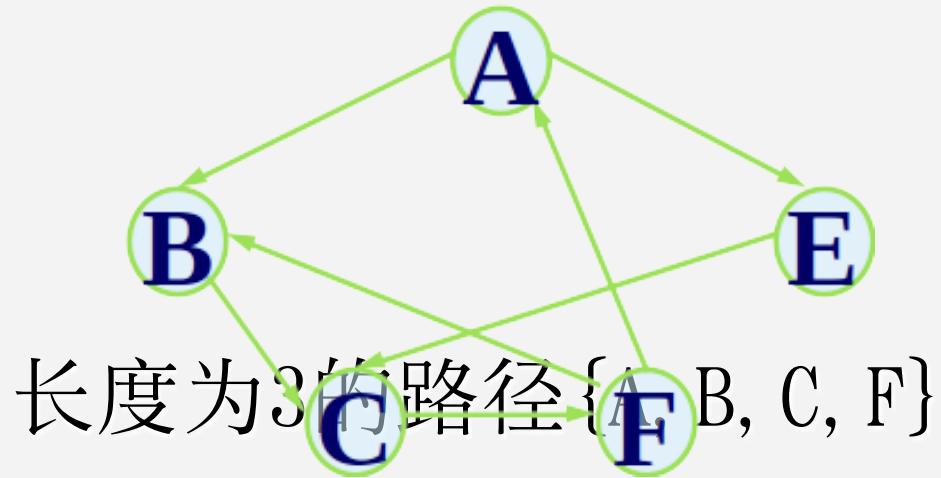
- 例如：图G1中顶点 v_1 的入度是 $ID(v_1)=1$ ，出度 $OD(v_1)=2$ ，顶点 v_1 的度 $TD(v_1)=ID(v_1)+OD(v_1)=3$ 。一般地，若图G中有 n 个顶点， e 条边，则图中顶点的度与边的关系如下：

度之和除以2就是边的数量

$$e = \frac{\sum_{i=1}^n TD(v_i)}{2}$$

7.1 图的定义和术语

- 无向图 $G=(V, \{E\})$ 中从顶点 v 到 v' 的路径是一个顶点序列 $v_0, v_1, v_2, \dots, v_n$, 其中 $(v_{i-1}, v_i) \in E$, $1 \leq i \leq n$ 。如果图 G 是有向图, 则路径也是有向的, 顶点序列应满足 $\langle v_{i-1}, v_i \rangle \in A$, $1 \leq i \leq n$ 。路径的长度是指路径上经过的边的数目。



7.1 图的定义和术语

- 在一个路径中，若其第一个顶点和最后一个顶点是相同的，即 $v=v'$ ，则称该路径为一个回路或环。
- 若表示路径的顶点序列中的顶点各不相同，则称这样的路径为简单路径。除了第一个和最后一个顶点外，其余各顶点均不重复出现的回路为简单回路或简单环。

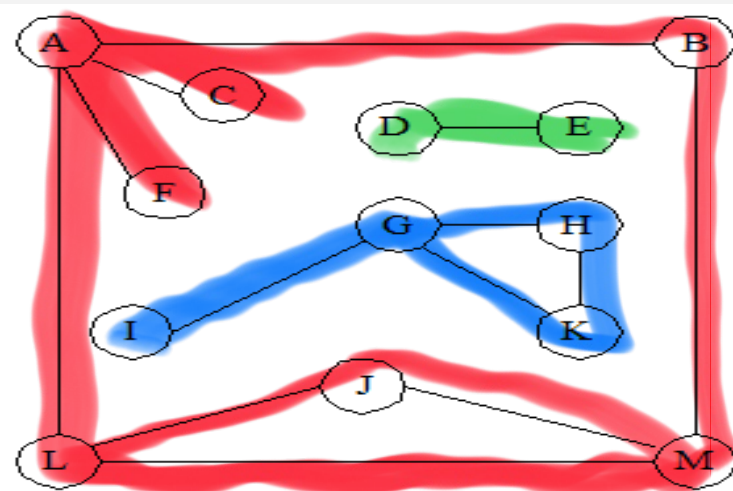
7.1 图的定义和术语

- 在无向图 $G=(V, \{E\})$ 中，若从 v_i 到 v_j 有路径相通，则称顶点 v_i 与 v_j 是连通的。
- 如果对于图中的任意两个顶点 $v_i, v_j \in V$ ， v_i, v_j 都是连通的，则称该无向图 G 为连通图。
- 例如： G_2 就是连通图。
- 无向图中的极大连通子图称为该无向图的连通分量。

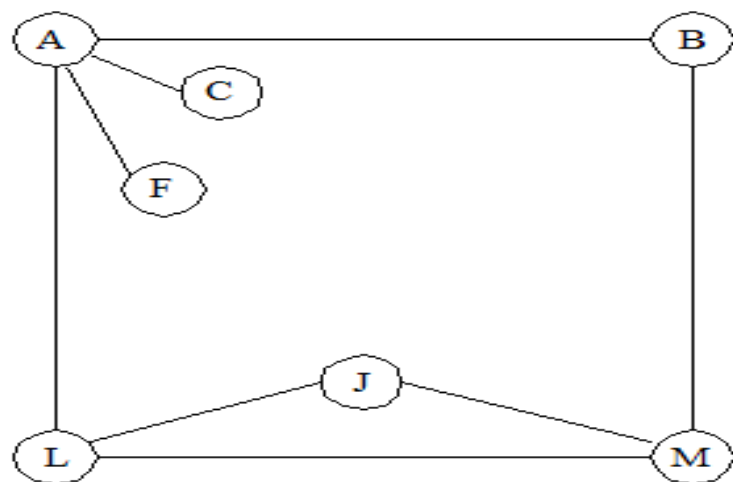
7.1 图的定义和术语

- 在有向图 $G=(V, \{A\})$ 中，若对于任意两个不同的顶点 $u, v \in V$ ，都存在从 u 到 v 的路径，则称该有向图为**强连通图**。
- 有向图的**极大强连通子图**称作有向图的**强连通分量**。

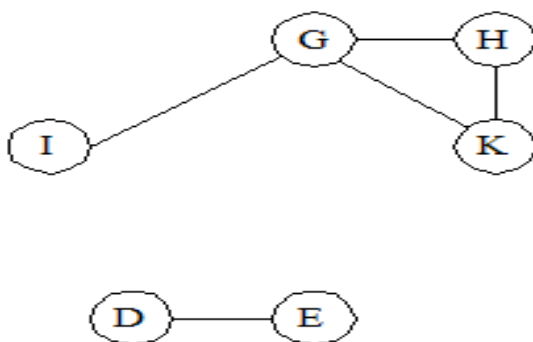
7.1 图的定义和术语



(a) 无向图 G_3



(b) 无向图 G_3 的三个连通分量



(c) 有向图 G_1 的两个连通分量

7.1 图的定义和术语

- 假设一个连通图有 n 个顶点和 e 条边，其中 $n-1$ 条边和 n 个顶点构成一个极小连通子图，称该极小连通子图为此连通图的生成树。
- 如果在一棵生成树上添加一条边，必定构成一个简单环。

↓
生成树不是唯一的

7.1 图的定义和术语

- 一棵有 n 个顶点的生成树有且仅有 $n-1$ 条边。如果一个图有 n 个顶点和小于 $n-1$ 条边，则是非连通图；如果多于 $n-1$ 条边，则一定有环。
- 对非连通图，则称由各个连通分量的生成树的集合为此非连通图的生成森林。

7.1 图的定义和术语

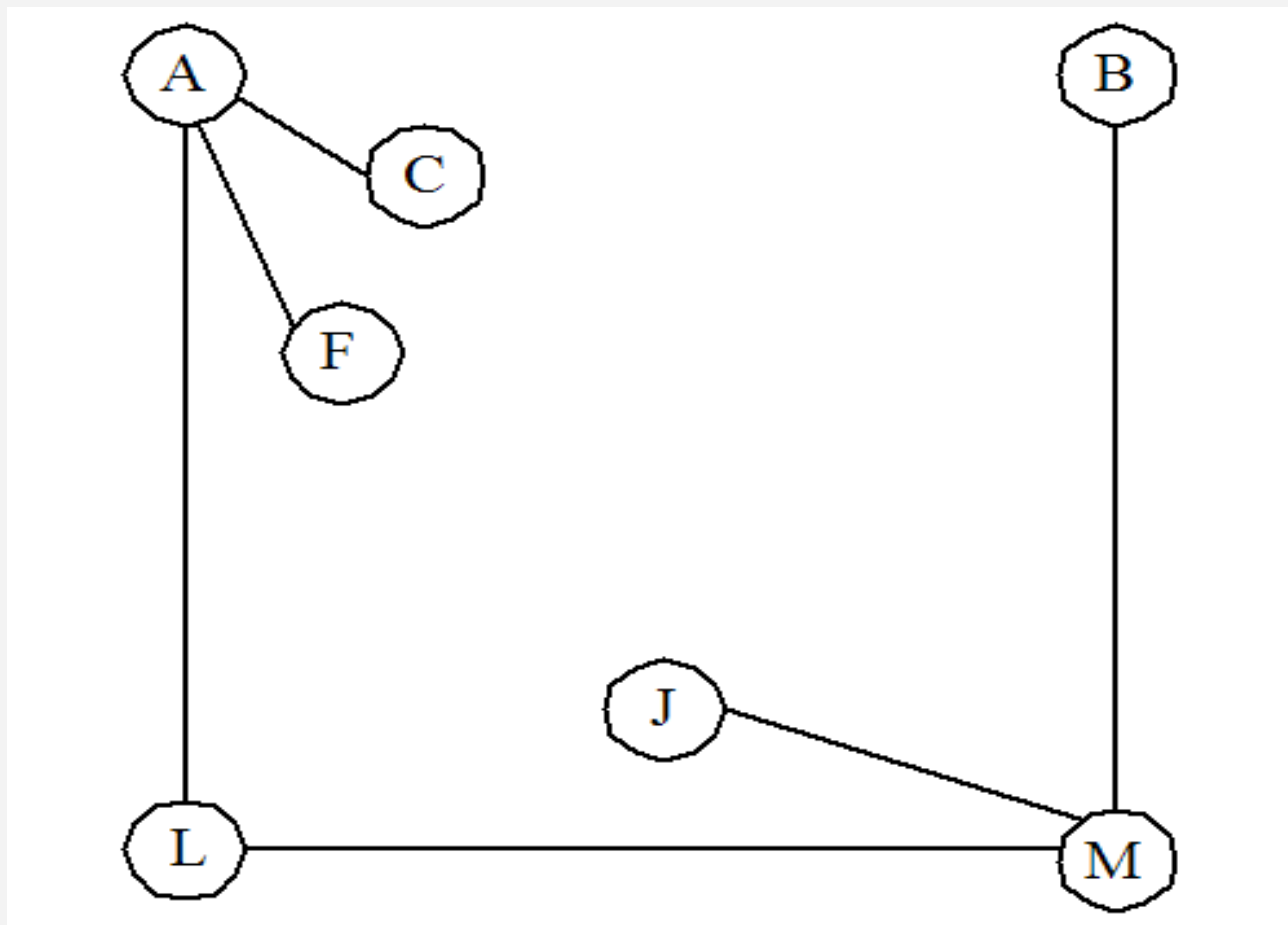


图 G_3 的最大连通分量的一棵生成树

7.2 图的存储表示

- 由于图的结构比较复杂，任意两个顶点之间都可能存在联系，因此无法以数据元素在存储区中的物理位置来表示元素之间的关系，即图没有顺序映像的存储结构。
- 常用的存储结构包括数组表示（邻接矩阵）和邻接表。

7.2.1 邻接矩阵

- 图的邻接矩阵表示法 (Adjacency Matrix) 也称作数组表示法。它采用两个数组来表示图。
- 一个是用于存储顶点信息的一维数组；另一个是用于存储图中顶点之间关联关系的二维数组，这个关联关系数组被称为邻接矩阵。
- 若图G是一个具有n个顶点的无权图，G的邻接矩阵是具有如下性质的 $n \times n$ 矩阵A:

$$A[i, j] = \begin{cases} 1 & \text{若 } \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in VR \\ 0 & \text{反之} \end{cases}$$

7.2.1 邻接矩阵

无向图的邻接矩阵是对称矩阵，有向图的邻接矩阵一般为非对称矩阵

$$A1 = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

$$A2 = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}$$

图G1， G2的邻接矩阵

7.2.1 邻接矩阵

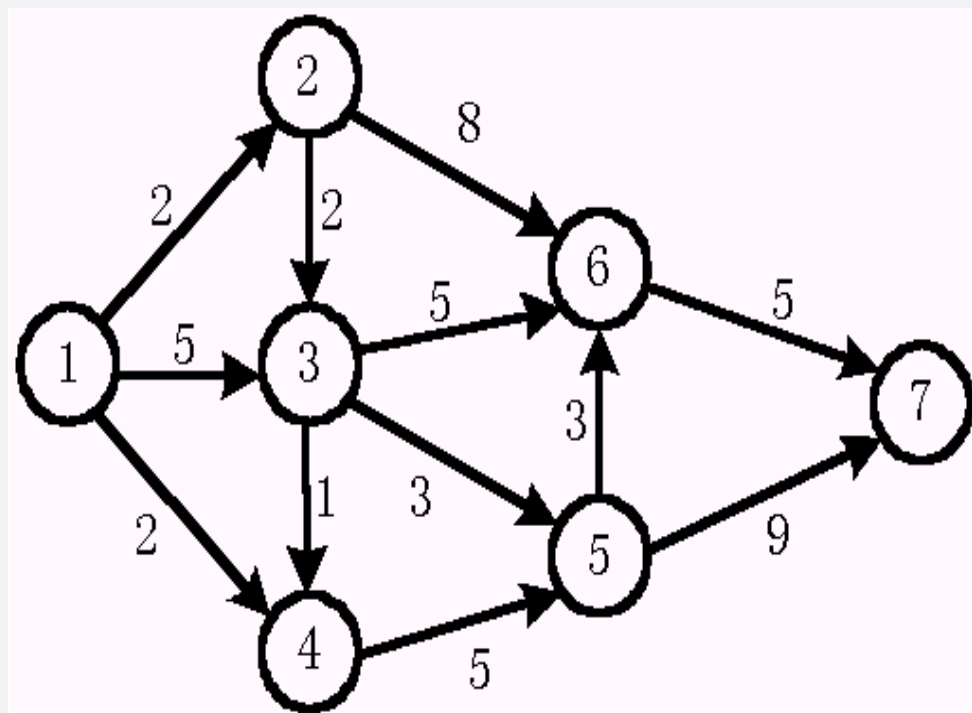
- 在无向图中, 统计第 i 行(列)1的个数可得顶点 v_i 的度。
- 在有向图中, 统计第 i 行1的个数可得顶点 v_i 的出度, 统计第 j 列1的个数可得顶点 v_j 的入度。

7.2.1 邻接矩阵

- 若图G是一个有n个顶点的网，则它的邻接矩阵是具有如下性质的 $n \times n$ 矩阵A:

$$A[i, j] = \begin{cases} w_{ij} & \text{若 } \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in VR \\ \infty & \text{反之} \end{cases}$$

7.2.1 邻接矩阵


$$\begin{bmatrix} \infty & 2 & 5 & 2 & \infty & \infty & \infty \\ \infty & \infty & 2 & \infty & \infty & 8 & \infty \\ \infty & \infty & \infty & 1 & 3 & 5 & \infty \\ \infty & \infty & \infty & \infty & 5 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 3 & 9 \\ \infty & \infty & \infty & \infty & \infty & \infty & 5 \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty \end{bmatrix}$$

7.2.1 邻接矩阵

邻接矩阵的特点

1. 存储空间：对于无向图而言，它的邻接矩阵是对称矩阵，因此可以采用特殊矩阵的压缩存储法，即只存储其下三角。一个具有 n 个顶点的无向图 G ，它的邻接矩阵需要 $n(n-1)/2$ 个存储空间即可。
2. 对于稀疏图而言，不适于用邻接矩阵来存储，因为这样会造成存储空间的浪费。
3. 便于运算：采用邻接矩阵表示法，便于判定图中任意两个顶点之间是否有边相连，即根据 $A[i, j] = 0$ 或 1 来判断。另外还便于求得各个顶点的度。

7.2.1 邻接矩阵

对于无向图而言，其邻接矩阵第*i*行元素之和就是图中第*i*个顶点的度：

$$TD(v_i) = \sum_{j=1}^n A[i, j]$$

对于有向图而言，其邻接矩阵第*i*行元素之和就是图中第*i*个顶点的出度：

$$OD(v_i) = \sum_{j=1}^n A[i, j]$$

对于有向图而言，其邻接矩阵第*i*列元素之和就是图中第*i*个顶点的入度：

$$ID(v_i) = \sum_{j=1}^n A[j, i]$$

7.2.2 邻接表

- 图的邻接矩阵表示法(即图的数组表示法), 虽然有其自身的优点, 但对于稀疏图来讲, 用邻接矩阵的表示方法会造成存储空间的很大浪费。
- 邻接表 (Adjacency List) 表示法实际上是图的一种链式存储结构。
- 它克服了邻接矩阵的弊病, 基本思想是只存有关联的信息, 对于图中存在的边信息则存储, 而对于不相邻的顶点则不保留信息。

7.2.2 邻接表

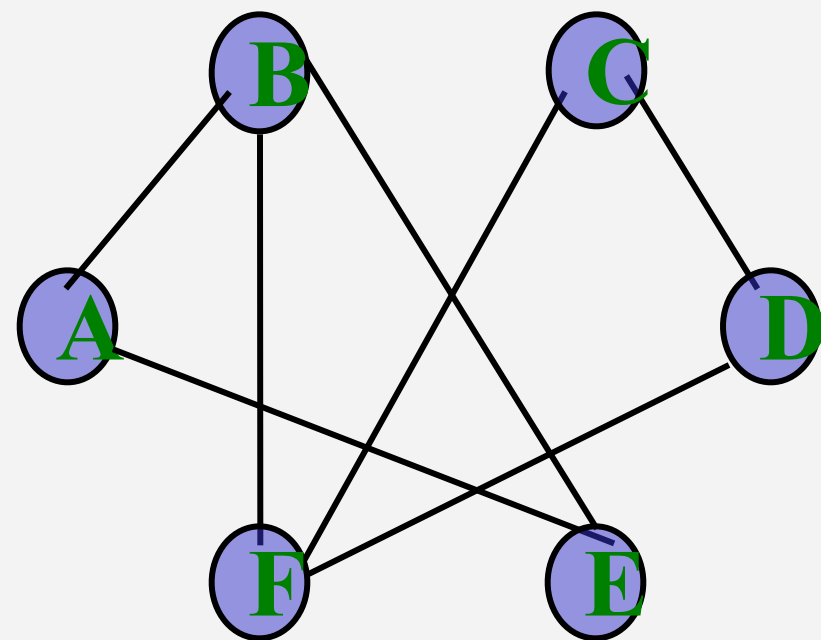
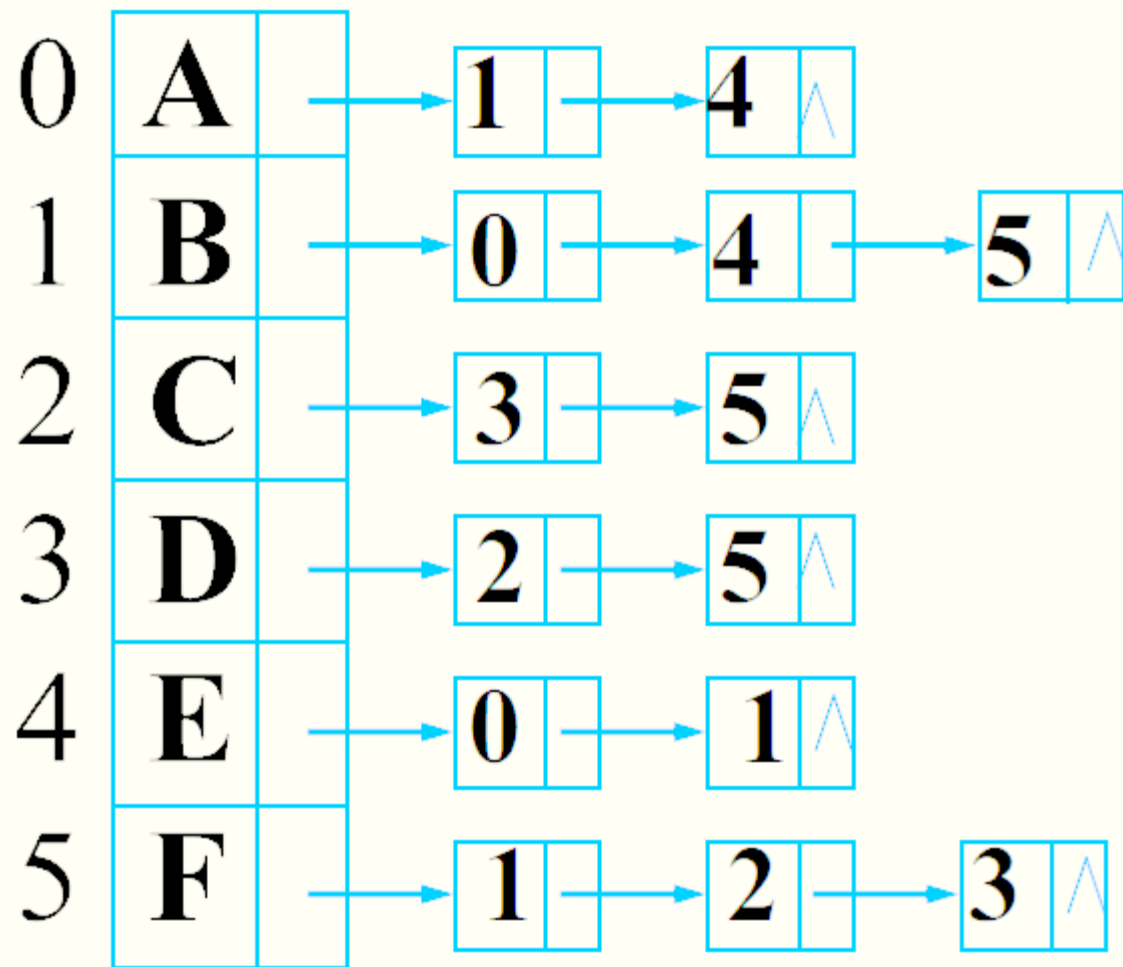
- 在邻接表中，对图中的每个顶点建立一个带头结点的单链表，如第 i 个单链表中的结点则表示依附于顶点 v_i 的边（若是有向图，则表示以 v_i 为起点的边）。每个单链表的头结点又构成一个表头结点表。这样，一个 n 个顶点的图的邻接表表示由表头结点表与单链表两部分构成。
- 为了简单起见，也可以不含头结点。

7.2.2 邻接表

图的邻接表存储结构

- 顶点以顺序结构存储，其数据域data存放顶点信息，链域adjlink存储指向由顶点相关联的边组成的链表。
- 把同一个顶点发出的边链接在同一个单链表中，链表的每一个结点代表一条边，叫做表结点（边结点）。

7.2.2 邻接表



7.2.2 邻接表

网的邻接表存储结构：由于每条边还包含有权值，因此表示边的邻接链表中，边的数据信息还多一项权域。

7.2.2 邻接表

- 存储空间：对于有 n 个顶点， e 条边的无向图而言，若采取邻接表作为存储结构，则需要 n 个表头结点和 $2e$ 个表结点。
- 很显然在边很稀疏（即 e 远小于 $n(n-1)/2$ 时）的情况下，用邻接表存储所需的空间比邻接矩阵所需的空间 $(n(n-1)/2)$ 要节省得多。

7.2.2 邻接表

- 无向图的度：在无向图的邻接表中，顶点 v_i 的度恰好就是第 i 个边链表上结点的个数。
- 有向图的度：在有向图中，第 i 个边链表上顶点的个数是顶点 v_i 的出度，只需通过表头向量表中找到第 i 个顶点的边链表的头指针，实现顺链查找即可。

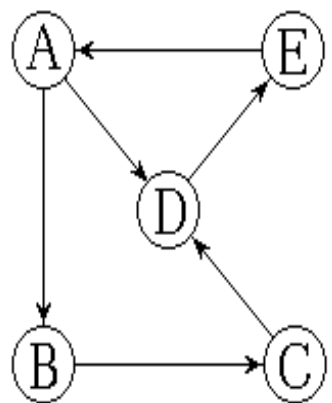
7.2.2 邻接表

- 对于有向图，如要判定任意两个顶点(v_i 和 v_j)之间是否有边相连，则需要搜索所有的单链表，这样比较麻烦。
- 求得第 i 个顶点的入度，也必须遍历整个邻接表，在所有边链表中查找邻接点域的值为 i 的结点并计数求和。

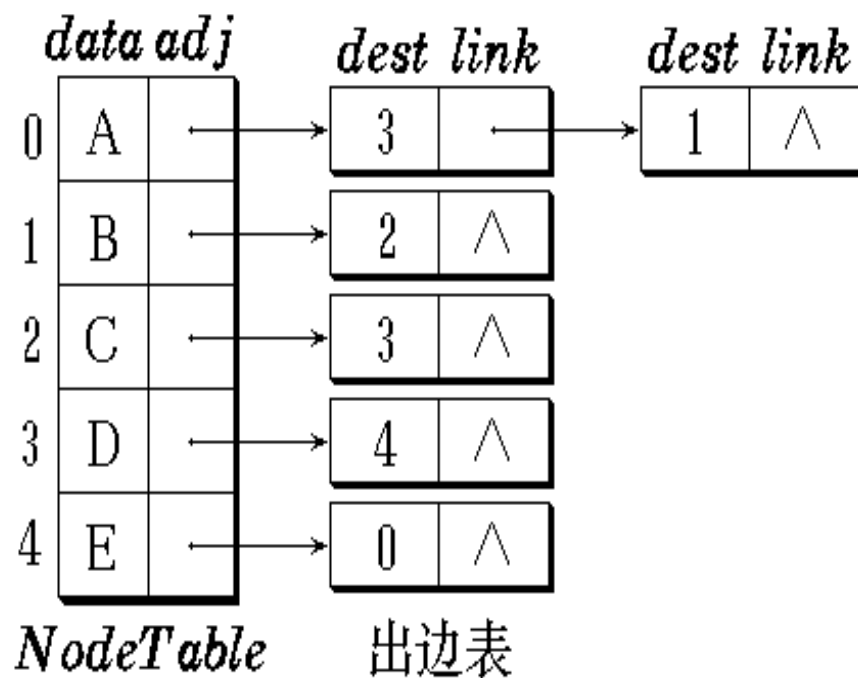
7.2.2 邻接表

- 由此可见，对于用邻接表方式存储的有向图，求顶点的入度并不方便，它需要通过扫描整个邻接表才能得到结果。
- 一种解决的方法是逆邻接表法，我们可以对每一顶点 v_i 再建立一个逆邻接表，即对每个顶点 v_i 建立一个所有以顶点 v_i 为终点的表。

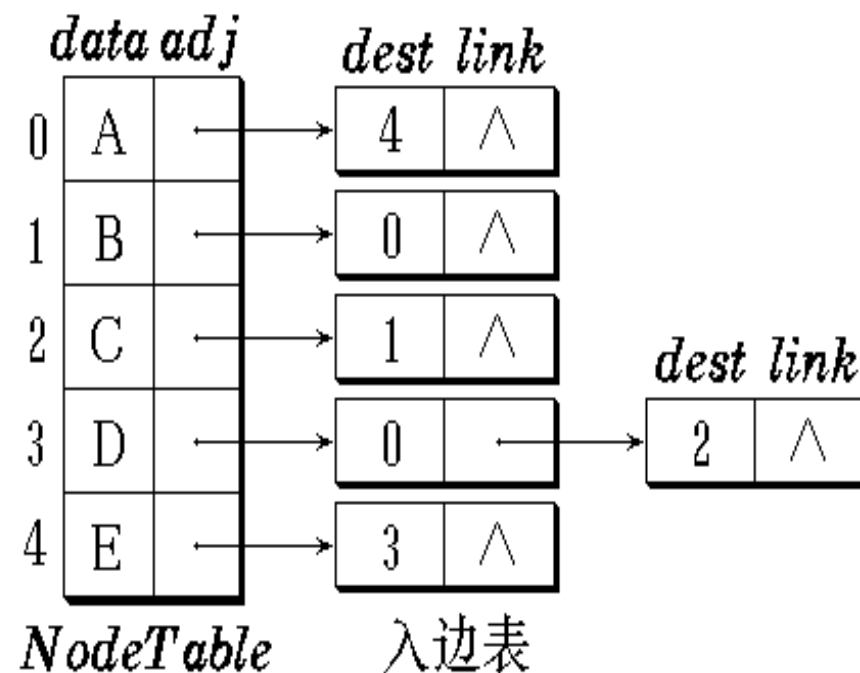
7.2.2 邻接表



G7



(a) 邻接表



(b) 逆邻接表

在有向图的邻接表中，第 i 个链表中结点的个数是顶点 V_i 的出度。
在有向图的逆邻接表中，第 i 个链表中结点的个数是顶点 V_i 的入度。

练习

设图的邻接矩阵A如下所示。各顶点的度依次是

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

A. 1, 2, 1, 2

C. 3, 4, 2, 3

B. 2, 2, 1, 1

D. 4, 4, 2, 2

答案 C

解析：各顶点的度是矩阵中此结点对应的横行和纵列非零元素之和。

练习

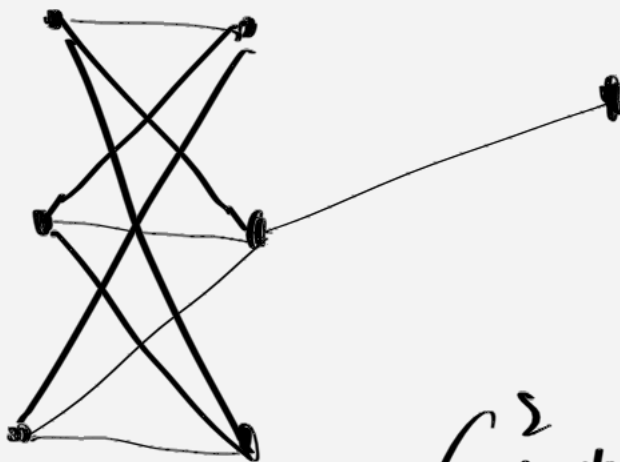
若无向图 $G(V, E)$ 中含7个顶点，则保证图 G 在任何情况下都是连通的，则需要的边数最少是 (C)

A : 6

B: 15

C: 16

D: 21



$$C_6^2 + 1 = \frac{6!}{2! \times 4!} + 1 = 16$$

7.3 图的遍历

- 从图中某一顶点出发访遍图中其余顶点，且使每个顶点仅被访问一次，就叫做图的遍历。
- 图的遍历算法是求解图的连通性问题、拓扑排序和求关键路径等算法的基础。
- 两条遍历图的路径：深度优先搜索、广度优先搜索。

7.3.1 深度优先搜索

- 深度优先搜索（Depth-First Search）是指按照深度方向搜索，它类似于树的先根遍历，是树的先根遍历的推广。

7.3.1 深度优先搜索

深度优先搜索连通子图的基本思想是：

- (1) 从图中某个顶点 v_0 出发，首先访问 v_0 。
- (2) 找出刚访问过的顶点 v_i 的第一个未被访问的邻接点，然后访问该顶点。以该顶点为新顶点，重复本步骤，直到当前的顶点没有未被访问的邻接点为止。
- (3) 返回前一个访问过的且仍有未被访问的邻接点的顶点，找出并访问该顶点的下一个未被访问的邻接点，然后执行步骤(2)。

7.3.1 深度优先搜索

采用递归的形式说明，则深度优先搜索连通子图的基本思想可表示为

- (1) 访问出发点 v_0
- (2) 依次以 v_0 的未被访问的邻接点为出发点，深度优先搜索图，直至图中所有与 v_0 有路径相通的顶点都被访问。

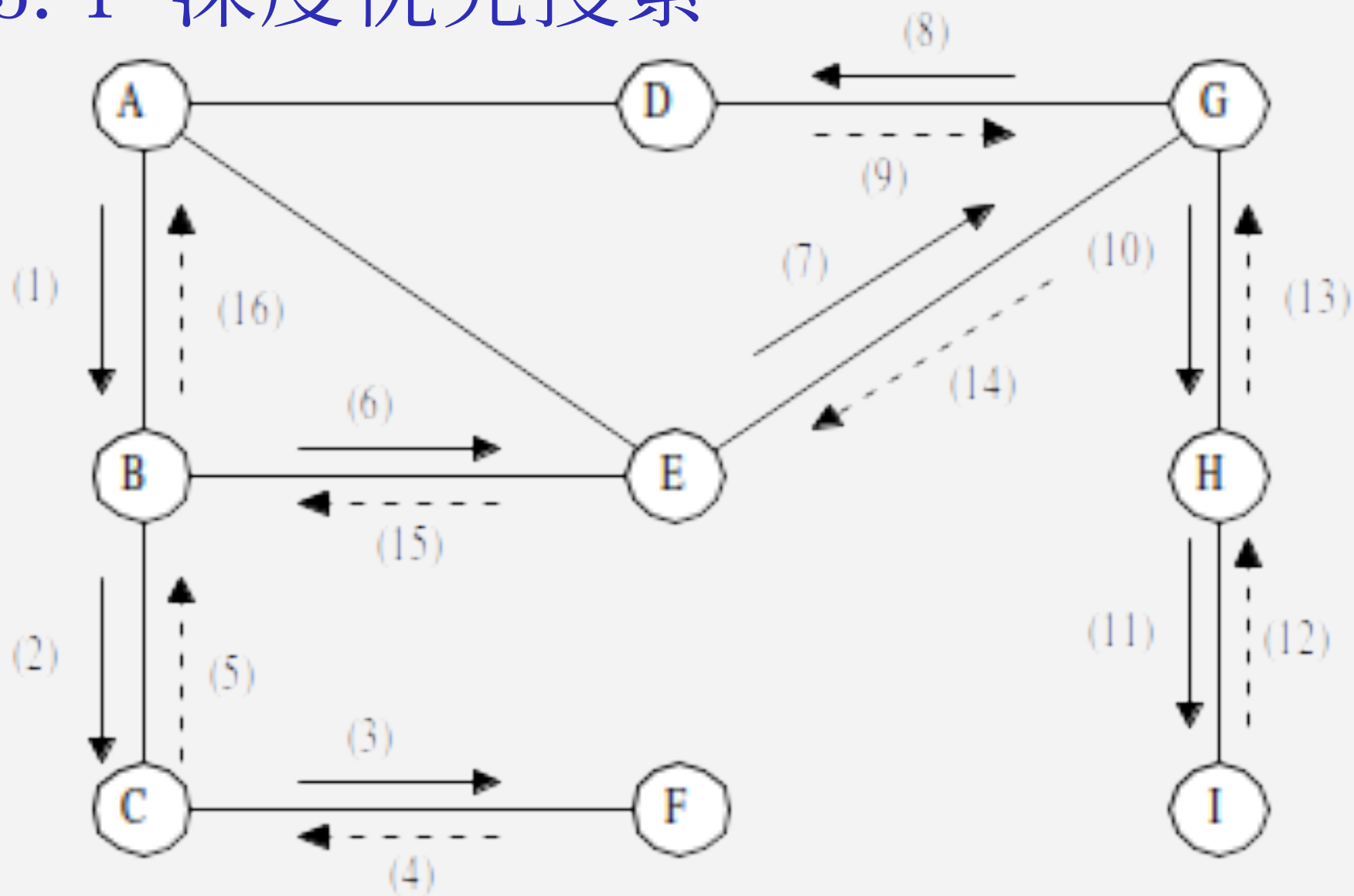
7.3.1 深度优先搜索

如何判别 v 的邻接点是否被访问：图的遍历算法一般为图的每个顶点保留一个标志域(tag)，值为UNVISITED表示没有被访问，值为VISITED表示已被访问。

7.3.1 深度优先搜索

非连通图的深度优先搜索遍历：首先将图中每个顶点的访问标志设为 UNVISITED，之后搜索图中每个顶点，如果未被访问，则以该顶点为起始点，进行深度优先搜索，否则继续检查下一顶点。

7.3.1 深度优先搜索



7.3.1 深度优先搜索

首先访问A，然后按图中序号对应的顺序进行深度优先搜索。图中序号对应步骤的解释如下： ☐

- (1) 顶点A的未访邻接点有B、E、D， 首先访问A的第一个未访邻接点B；
- (2) 顶点B的未访邻接点有C、E， 首先访问B的第一个未访邻接点C；
- (3) 顶点C的未访邻接点只有F， 访问F； ☐
- (4) 顶点F没有未访邻接点， 回溯到C； ☐
- (5) 顶点C已没有未访邻接点， 回溯到B；
- (6) 顶点B的未访邻接点只剩下E， 访问E；

7.3.1 深度优先搜索

首先访问A，然后按图中序号对应的顺序进行深度优先搜索。图中序号对应步骤的解释如下： ☐

(7) 顶点E的未访邻接点只剩下G，访问G； ☐

(8) 顶点G的未访邻接点有D、H，首先访问G的第一个未访邻接点D；

(9) 顶点D没有未访邻接点，回溯到G； ☐

(10) 顶点G的未访邻接点只剩下H，访问H； ☐

(11) 顶点H的未访邻接点只有I，访问I； ☐

(12) 顶点I没有未访邻接点，回溯到H； ☐

7.3.1 深度优先搜索

首先访问A，然后按图中序号对应的顺序进行深度优先搜索。图中序号对应步骤的解释如下： ☐

(13) 顶点H已没有未访邻接点， 回溯到G； ☐

(14) 顶点G已没有未访邻接点， 回溯到E； ☐

(15) 顶点E已没有未访邻接点， 回溯到B； ☐

(16) 顶点B已没有未访邻接点， 回溯到A。

7.3.1 深度优先搜索

算法分析

- 图中有 n 个顶点， e 条边
- 如果用邻接表表示图，沿 `Firstarc` à `nextarc` 链可以找到某个顶点 v 的所有邻接顶点 w 。由于总共有 $2e$ 个边结点，所以扫描边的时间为 $O(e)$ 。而且对所有顶点递归访问1次，所以遍历图的时间复杂度为 $O(n+e)$
- 如果用邻接矩阵表示图，则查找每一个顶点的所有的边，所需时间为 $O(n)$ ，则遍历图中所有的顶点所需的时间为 $O(n^2)$

7.3.1 深度优先搜索

```
/**
 * @brief 递归版DFS (从顶点v开始遍历)
 * @param G 图 (值传递, 不修改原图)
 * @param v 起始顶点索引
 * @param visited 访问标记数组 (1=已访问, 0=未访问)
 */
void DFS_Recursive(ALGraph G, int v, int visited[]) {
    ArcNode *p;

    visited[v] = 1; // 标记当前顶点为已访问
    printf("%c ", G.vertices[v].data); // 访问顶点 (输出数据)

    p = G.vertices[v].firstarc; // 指向当前顶点的第一个邻接顶点
    while (p != NULL) {
        if (!visited[p->adjvex]) { // 若邻接顶点未访问
            DFS_Recursive(G, p->adjvex, visited); // 递归访问
        }
        p = p->nextarc; // 遍历下一个邻接顶点
    }
}
```

7.3.1 深度优先搜索

```
void DFS_Iterative(ALGraph G, int v, int visited[]) {
    ArcNode *p;
    int stack[100], top = -1; // 栈: 存储顶点索引, top为栈顶指针 (-1表示空栈)

    stack[++top] = v; // 起始顶点入栈
    visited[v] = 1;   // 标记为已访问

    while (top != -1) { // 栈不为空时循环
        v = stack[top--]; // 出栈 (当前访问顶点)
        printf("%c ", G.vertices[v].data); // 访问顶点

        // 邻接顶点入栈 (需逆序, 保证与递归版遍历顺序一致)
        int temp_stack[100], temp_top = -1; // 临时栈: 反转邻接顶点顺序
        p = G.vertices[v].firstarc;
        while (p != NULL) {
            if (!visited[p->adjvex]) {
                visited[p->adjvex] = 1;
                temp_stack[++temp_top] = p->adjvex; // 先存入临时栈
            }
            p = p->nextarc;
        }

        // 临时栈元素倒入主栈 (反转顺序)
        while (temp_top != -1) {
            stack[++top] = temp_stack[temp_top--];
        }
    }
}
```

7.3.2 广度优先搜索

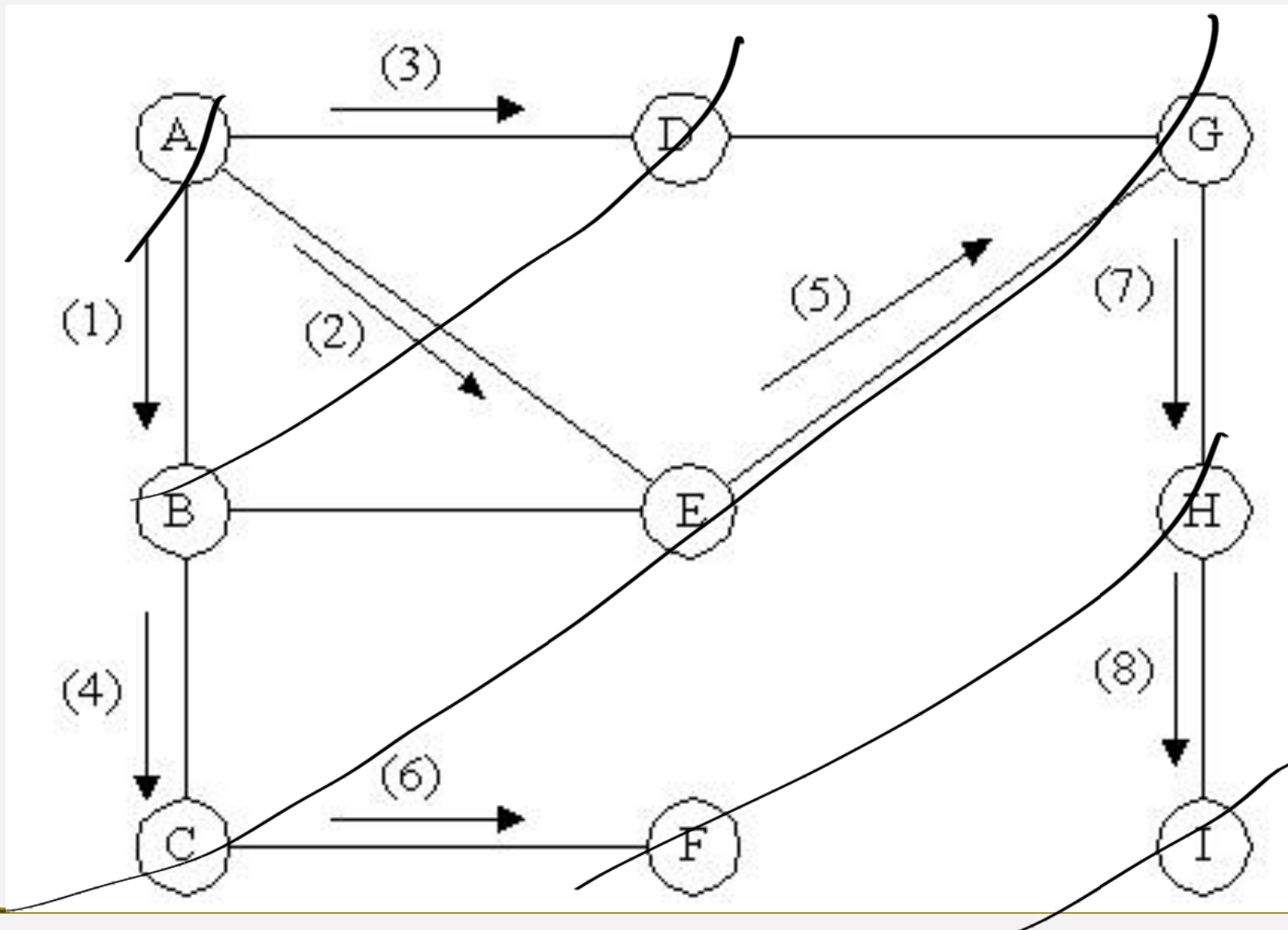
广度优先搜索（Breadth-First Search）是指按照广度方向搜索，它类似于树的层次遍历，是树的按层次遍历的推广。

7.3.2 广度优先搜索

广度优先搜索的基本思想是：

- (1) 从图中某个顶点 v_0 出发，首先访问 v_0 。
- (2) 依次访问 v_0 的各个未被访问的邻接点。
- (3) 分别从这些邻接点出发，依次访问它们的各个未被访问的邻接点。访问时应保证：如果 v_i 和 v_k 为新的邻接结点， v_i 在 v_k 之前被访问，则 v_i 的所有未被访问的邻接点应在 v_k 的所有未被访问的邻接点之前访问。重复（3），直到所有邻接点均没有未被访问的邻接点为止。

7.3.2 广度优先搜索



7.3.2 广度优先搜索

```
void BFS(ALGraph G, int start, int visited[]) {
    ArcNode *p;
    int queue[100]; // 队列: 存储顶点索引 (最大100个顶点)
    int front = 0, rear = 0; // 队列头尾指针 (front=队头, rear=队尾+1, 空队: front==rear)

    // 起始顶点入队并标记
    queue[rear++] = start;
    visited[start] = 1;

    // 队列非空时循环
    while (front < rear) {
        int v = queue[front++]; // 出队 (取队头元素)
        printf("%c ", G.vertices[v].data); // 访问当前顶点

        // 遍历当前顶点的所有邻接顶点
        p = G.vertices[v].firstarc;
        while (p != NULL) {
            if (!visited[p->adjvex]) { // 邻接顶点未访问
                visited[p->adjvex] = 1; // 标记
                queue[rear++] = p->adjvex; // 入队
            }
            p = p->nextarc; // 下一个邻接顶点
        }
    }
}
```


练习

- 5、对有 n 个结点、 e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法时间复杂度是
- A. $O(n)$ B. $O(e)$ C. $O(n+e)$ D. $O(n*e)$

参考答案：C

(注意掌握深度优先和广度优先在不同存储结构下的时间复杂度)

练习

若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是

A. h, c, a, b, d, e, g, f ✓

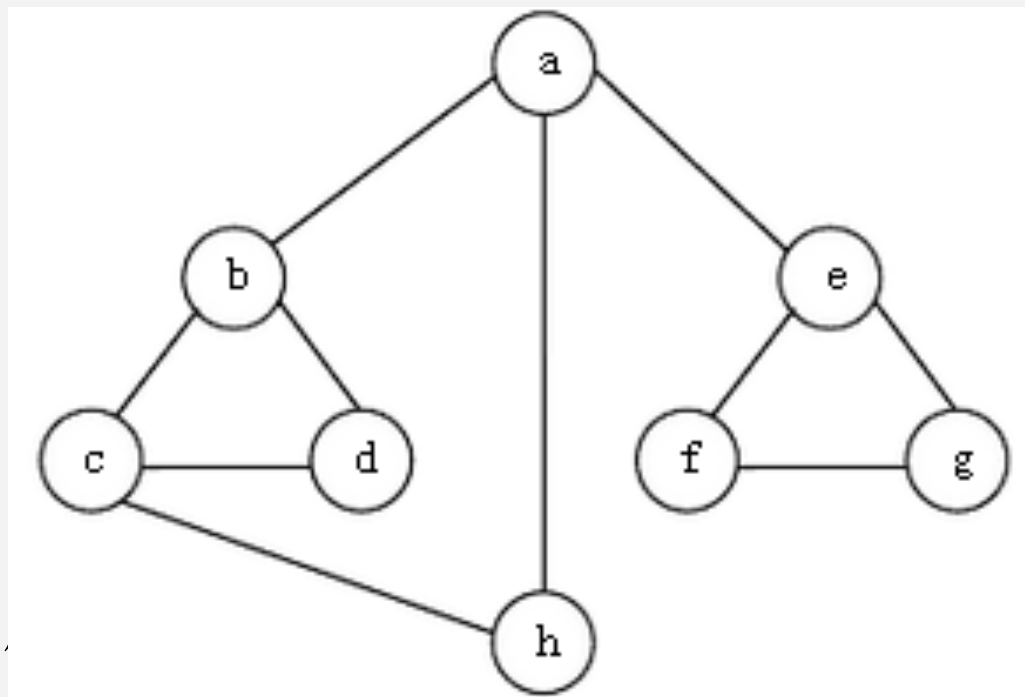
B. e, a, f, g, b, h, c, d

C. d, b, c, a, h, e, f, g

D. a, b, c, d, h, e, f, g

答案 D

解析：D选项是深度优先遍历不是广



~~a b e c d h f g~~
a b e h c d f g

7.4 图的最小代价生成树

- 对无向连通图进行遍历时，仅需从图中任意一顶点出发，进行搜索便可以访问所有顶点。
- 对非连通图，则需从多个顶点出发进行搜索，而每一次从一个新的起点出发进行搜索过程中得到的顶点访问序列恰好为其各个连通分量中的顶点集。

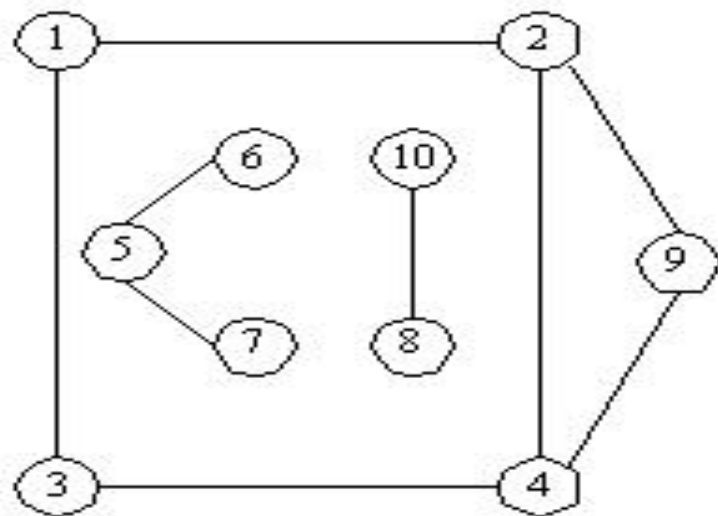
7.4 图的最小代价生成树

- 假设 $E(G)$ 为连通图 G 中所有边的集合，则从图中任一顶点出发遍历图时，必将 $E(G)$ 分成两个集合 $T(G)$ 和 $B(G)$ ，其中 $T(G)$ 是遍历过程中历经的边的集合； $B(G)$ 是剩余的边的集合。则边集 $T(G)$ 和图 G 中所有顶点一起构成连通图 G 的一棵生成树。

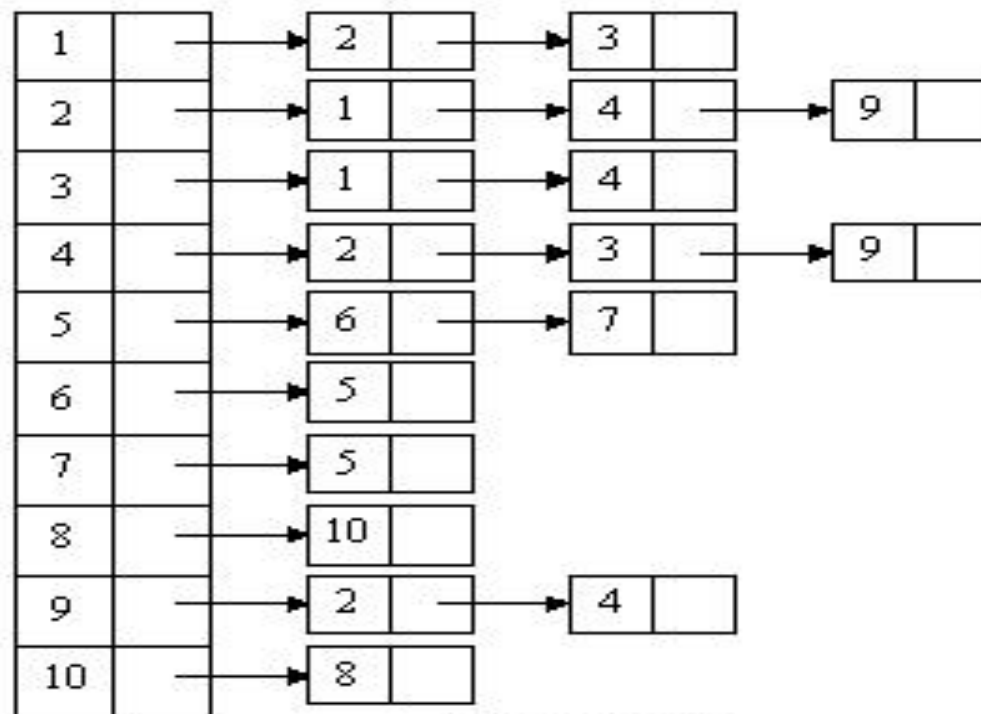
7.4 图的最小代价生成树

- 使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树。如深度优先生成树、广度优先生成树。
- 对于非连通图，每个连通分量中的顶点集和遍历时走过的边一起构成若干棵生成树，这些连通分量的生成树组成非连通图的生成森林。

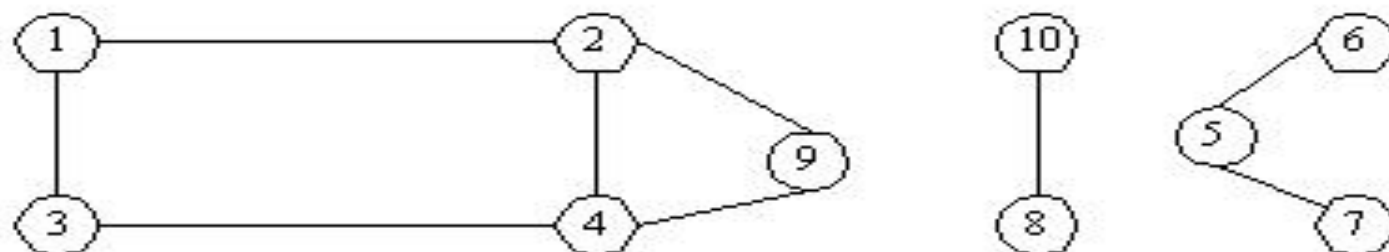
7.4 图的最小代价生成树



(a) 无向图 G_5



(b) G_5 的邻接表



(c) 无向图 G_5 的三个连通分量

7.4 图的最小代价生成树

在一个连通网的所有生成树中，各边的代价之和最小的那棵生成树称为该连通网的最小代价生成树（Minimum Cost Spanning Tree），简称为最小生成树(MST)。

- 最小代价生成树边的条数是顶点个数减1，并且能保证最小生成树是连通的
- 最小生成树边上的权值之和最小

7.4 图的最小代价生成树

最小生成树有如下重要性质：设 $N=(V, \{E\})$ 是一连通网， U 是顶点集 V 的一个非空子集。若 (u, v) 是连接 U 与 $V-U$ 的一条具有最小权值的边，其中 $u \in U$, $v \in V-U$ ，则存在一棵包含边 (u, v) 的最小生成树。

7.4 图的最小代价生成树

用反证法来证明：假设不存在这样一棵包含边 (u, v) 的最小生成树。任取一棵最小生成树 T ，将 (u, v) 加入 T 中。根据树的性质，此时 T 中必形成一个包含 (u, v) 的回路，且回路中必有一条边 (u', v') 的权值大于或等于 (u, v) 的权值。删除 (u', v') ，则得到一棵代价小于等于 T 的生成树 T' ，且 T' 为一棵包含边 (u, v) 的最小生成树。这与假设矛盾，故该性质得证。

7.4.1 普里姆 (Prim) 算法

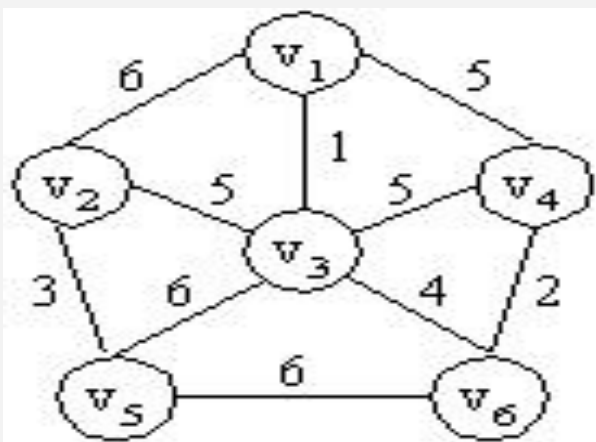
假设 $N=(V, \{E\})$ 是连通网， TE 为最小生成树中边的集合

- (1) 初始 $U=\{u_0\}$ ($u_0 \in V$), $TE=\phi$;
- (2) 在所有 $u \in U, v \in V-U$ 的边中选一条代价最小的边 (u_0, v_0) 并入集合 TE , 同时将 v_0 并入 U ;
- (3) 重复(2), 直到 $U=V$ 为止。
此时, TE 中必含有 $n-1$ 条边, 则 $T=(V, \{TE\})$ 为 N 的最小生成树。

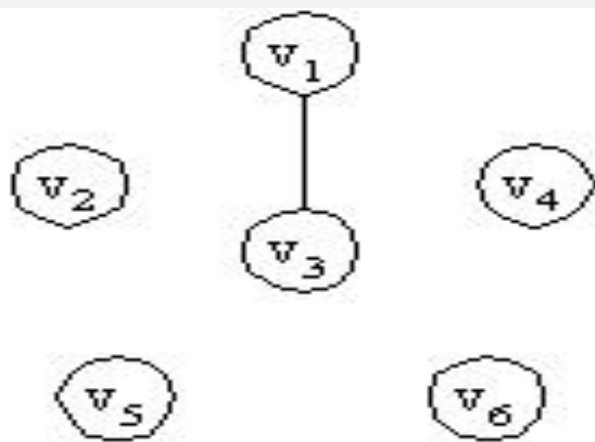
7.4.1 普里姆 (Prim) 算法

- 可以看出，普利姆算法逐步增加U中的顶点，可称为“加点法”。
- 为了实现这个算法需要设置一个辅助数组adjvex[]，用于存储从U到V-U具有最小权值的边。

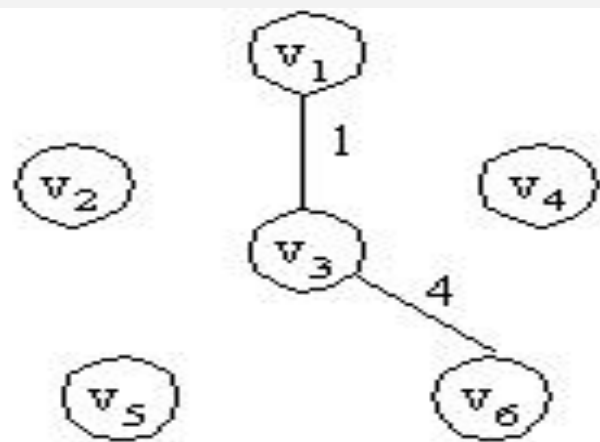
7.4.1 普里姆 (Prim) 算法



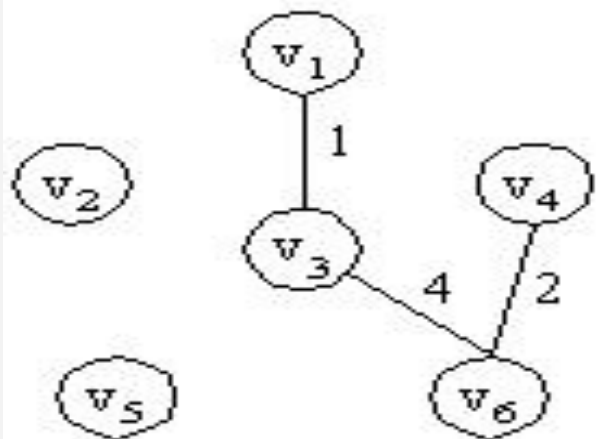
(a) 一个连通网



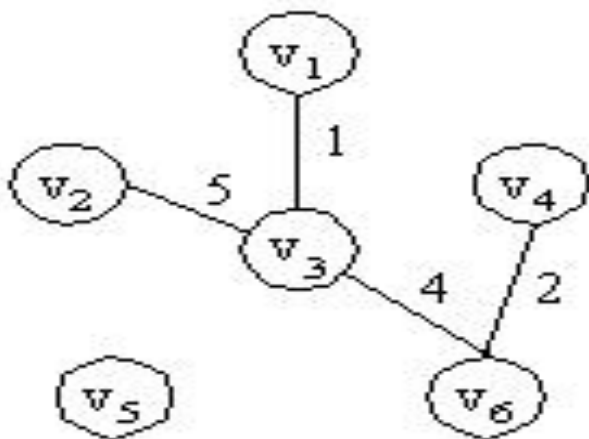
(b) 将 v_3 纳入 U 中



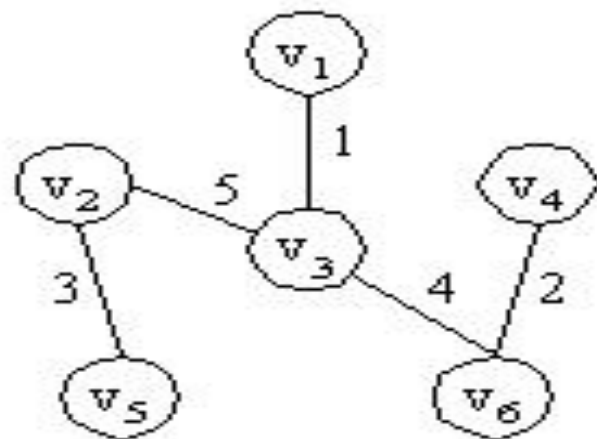
(c) 将 v_6 纳入 U 中



(d) 将 v_4 纳入 U 中



(e) 将 v_2 纳入 U 中



(f) 将 v_5 纳入 U 中

7.4.1 普里姆 (Prim) 算法

$\begin{array}{c} i \\ \text{Closededge}[i] \end{array}$	0	1	2	3	4	5	U	V-U	e	k_0	(u_0, v_0)
adjvex		v_1	v_1	v_1	v_1	v_1	$\{v_1\}$	$\{v_2, v_3, v_4, v_5, v_6\}$	1	2	(v_1, v_3)
lowcost	0	6	1	5	∞	∞					
adjvex		v_3		v_1	v_3	v_3	$\{v_1, v_3\}$	$\{v_2, v_4, v_5, v_6\}$	2	5	(v_3, v_6)
lowcost	0	5	0	5	6	4					
adjvex		v_3		v_6	v_3		$\{v_1, v_3, v_6\}$	$\{v_2, v_4, v_5\}$	3	3	(v_6, v_4)
lowcost	0	5	0	2	6	0					
adjvex		v_3			v_3		$\{v_1, v_3, v_6, v_4\}$	$\{v_2, v_5\}$	4	1	(v_3, v_2)
lowcost	0	5	0	0	6	0					
adjvex					v_2		$\{v_1, v_3, v_6, v_4, v_2\}$	$\{v_5\}$	5	4	(v_2, v_5)
lowcost	0	0	0	0	3	0					
adjvex							$\{v_1, v_3, v_6, v_4, v_2, v_5\}$	$\{\}$			
lowcost	0	0	0	0	0	0					

7.4.1 普里姆 (Prim) 算法

```
void Prim(ALGraph G, int start) {
    int lowcost[100];    // 存储顶点到生成树的最小权值 (0=已加入)
    int adjvex[100];     // 存储顶点连接到生成树的前驱顶点
    int total_weight = 0; // 最小生成树的总权值
    int i, j, k, min;
    ArcNode *p;

    // 步骤1: 初始化lowcost和adjvex数组
    for (i = 0; i < G.vexnum; i++) {
        // 初始时, lowcost设为start到i的直接边权 (无直接边则为INT_MAX)
        lowcost[i] = INT_MAX;
        adjvex[i] = start; // 前驱默认为起始顶点
    }

    // 起始顶点加入生成树 (lowcost[start] = 0标记)
    lowcost[start] = 0;
    p = G.vertices[start].firstarc;
    while (p != NULL) {
        j = p->adjvex;
        if (lowcost[j] > p->weight) { // 若start到j的边权更小
            lowcost[j] = p->weight;
        }
        p = p->nextarc;
    }
}
```

7.4.1 普里姆 (Prim) 算法

```
// 步骤2: 循环n-1次, 选择n-1条边
for (i = 1; i < G.vexnum; i++) {
    // 子步骤1: 找到lowcost中权值最小的未加入顶点k
    min = INT_MAX;
    k = -1;
    for (j = 0; j < G.vexnum; j++) {
        if (lowcost[j] != 0 && lowcost[j] < min) {
            min = lowcost[j];
            k = j;
        }
    }

    // 若k=-1, 说明图不连通, 无法生成最小生成树
    if (k == -1) {
        printf("图不连通, 无法生成最小生成树! \n");
        return;
    }

    // 子步骤2: 输出当前选择的边 (adjvex[k] → k) 及权值
    printf("边: %c - %c, 权值: %d\n",
        G.vertices[adjvex[k]].data,
        G.vertices[k].data,
        min);
    total_weight += min; // 累加总权值
```

7.4.1 普里姆 (Prim) 算法

```
// 子步骤3: 标记k为已加入生成树
lowcost[k] = 0;

// 子步骤4: 更新lowcost和adjvex (以k为新的生成树顶点)
p = G.vertices[k].firstarc;
while (p != NULL) {
    j = p->adjvex;
    // 若j未加入生成树, 且k到j的边权 < 当前lowcost[j]
    if (lowcost[j] != 0 && p->weight < lowcost[j]) {
        lowcost[j] = p->weight;
        adjvex[j] = k; // 更新前驱为k
    }
    p = p->nextarc;
}

// 步骤3: 输出最小生成树的总权值
printf("最小生成树的总权值: %d\n", total_weight);
}
```


7.4.2 克鲁斯卡尔 (Kruskal) 算法

假设 $N=(V, \{E\})$ 是连通网，将 N 中的边按权值从小到大的顺序排列

- (1) 将 n 个顶点看成 n 个集合；
- (2) 按权值由小到大的顺序选择边，所选边应满足两个顶点不在同一个顶点集合内，将该边放到生成树边的集合中。同时将该边的两个顶点所在的顶点集合合并；
- (3) 重复(2)，直到所有的顶点都在同一个顶点集合内。

可以看出，克鲁斯卡尔算法逐步增加生成树的边，与普里姆算法相比，可称为“加边法”。

7.4.2 克鲁斯卡尔 (Kruskal) 算法

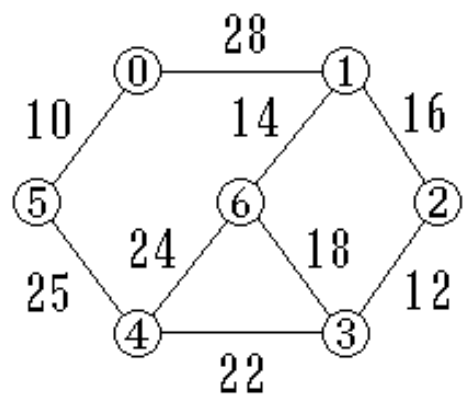
对于上图所示的连通网，将所有的边按权值从小到大的顺序排列为：

权值	1	2	3	4	5	5	5	6	6	6
边	(v_1, v_3)	(v_4, v_6)	(v_2, v_5)	(v_3, v_6)	(v_1, v_4)	(v_2, v_3)	(v_3, v_4)	(v_1, v_2)	(v_3, v_5)	(v_5, v_6)

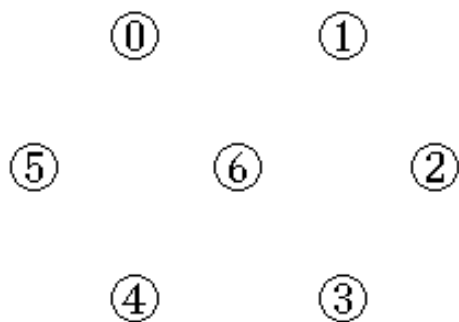
经过筛选所得到边的顺序为： (v_1, v_3) (v_4, v_6) (v_2, v_5) (v_3, v_6) (v_2, v_3)

在选择第五条边时，因为 v_1 、 v_4 已经在同一集合内，如果选 (v_1, v_4) ，则会形成回路，所以选 (v_2, v_3) 。

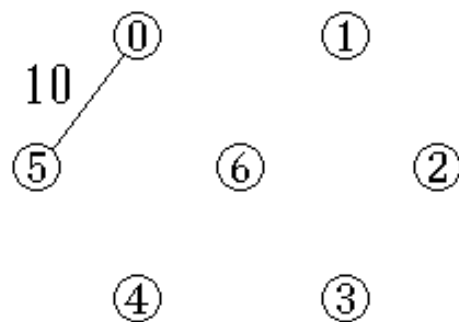
7.4.2 克鲁斯卡尔 (Kruskal) 算法



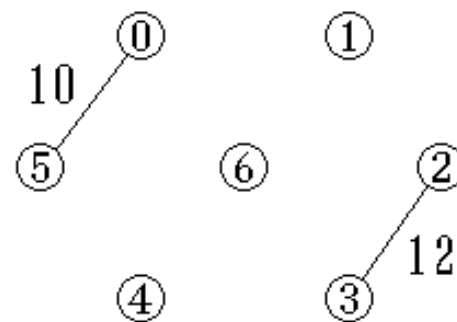
(a)



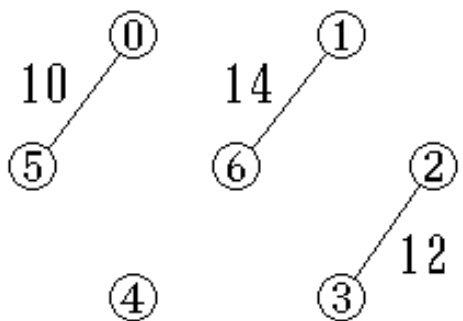
(b)



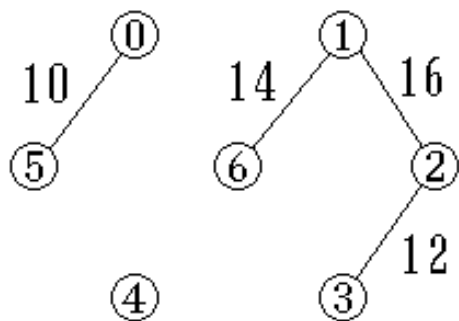
(c)



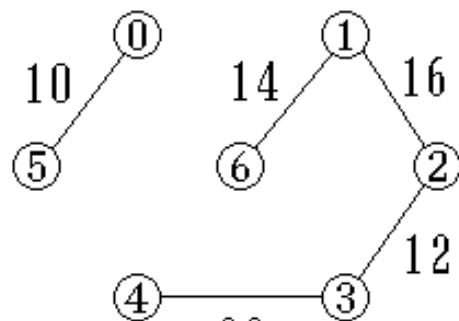
(d)



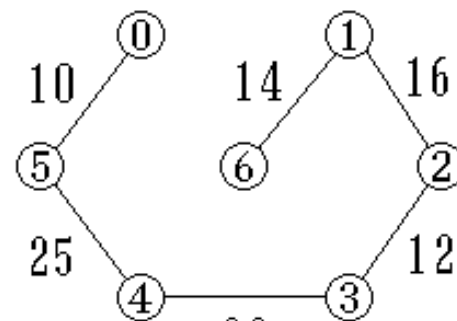
(e)



(f)

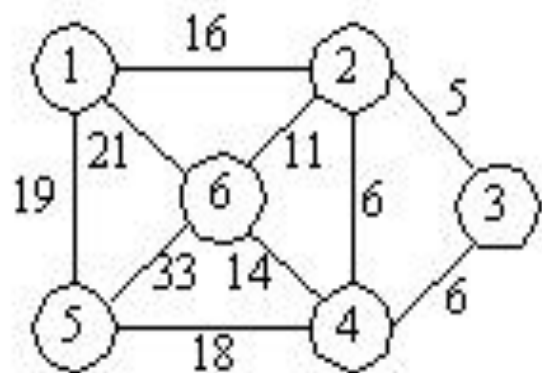


(g)

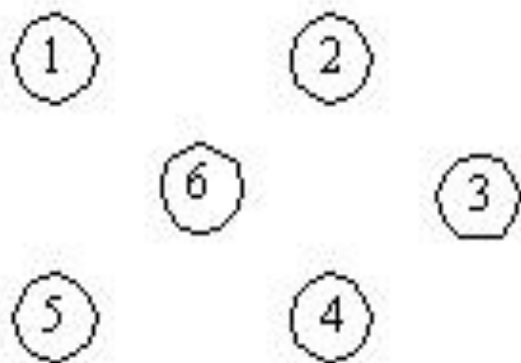


(h)

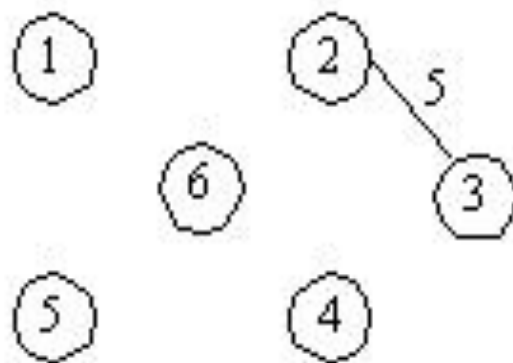
7.4.2 克鲁斯卡尔 (Kruskal) 算法



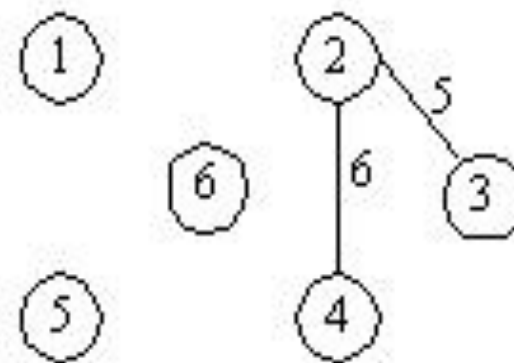
(a) 一个连通网



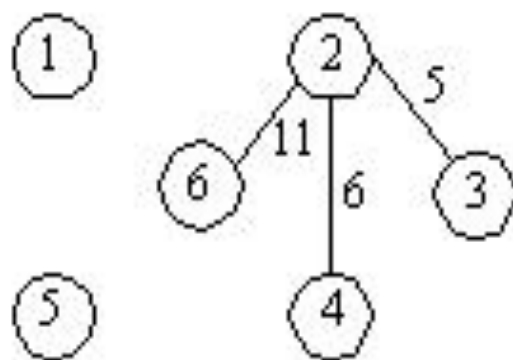
(b) 最小生成树的初始状态



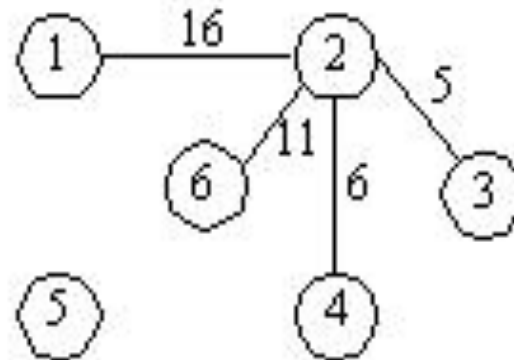
(c) 加入边(2, 3)



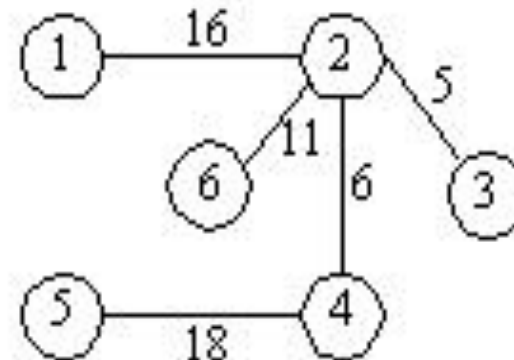
(d) 加入边(2, 4)



(e) 加入边(2, 6)



(f) 加入边(1, 2)



(g) 加入边(5, 4)

7.4.2 克鲁斯卡尔 (Kruskal) 算法

```
/**
 * 函数：克鲁斯卡尔算法求最小生成树
 * 参数：n-顶点数，m-边数，edges-边集数组，vertices-顶点数据数组（字符型）
 */
void Kruskal(int n, int m, Edge edges[], char vertices[]) {
    int edgeCount = 0; // 已加入生成树的边数
    int totalWeight = 0; // 最小生成树的总权值

    // 步骤1：初始化并查集
    for (int i = 0; i < n; i++) {
        parent[i] = i; // 每个顶点的父节点初始为自身
        rank_[i] = 0; // 初始秩为0
    }

    // 步骤2：对边集按权值升序排序
    qsort(edges, m, sizeof(Edge), compareEdge);
```

7.4.2 克鲁斯卡尔 (Kruskal) 算法

```
// 步骤3: 遍历排序后的边, 选择无环的边加入生成树
printf("克鲁斯卡尔算法求解最小生成树: \n");
for (int i = 0; i < m && edgeCount < n - 1; i++) {
    Edge currEdge = edges[i];
    int rootU = find(currEdge.u);
    int rootV = find(currEdge.v);

    if (rootU != rootV) { // 无环: u和v不在同一连通分量
        // 输出当前选中的边
        printf("边: %c - %c, 权值: %d\n",
            vertices[currEdge.u],
            vertices[currEdge.v],
            currEdge.weight);
        totalWeight += currEdge.weight;
        edgeCount++;
        unionSet(rootU, rootV); // 合并两个连通分量
    }
}

// 步骤4: 判断是否生成成功
if (edgeCount == n - 1) {
    printf("最小生成树的总权值: %d\n", totalWeight);
} else {
    printf("图不连通, 无法生成最小生成树! \n");
}
}
```

7.4.2 克鲁斯卡尔 (Kruskal) 算法

```
int find(int x) {  
    if (parent[x] != x) {  
        parent[x] = find(parent[x]); // 路径压缩: 直接指向根节点  
    }  
    return parent[x];  
}
```

```
void unionSet(int x, int y) {  
    int rootX = find(x);  
    int rootY = find(y);  
  
    if (rootX == rootY) {  
        return; // 已在同一连通分量, 无需合并  
    }  
  
    // 按秩合并: 秩小的树合并到秩大的树的根下  
    if (rank_[rootX] < rank_[rootY]) {  
        parent[rootX] = rootY;  
    } else if (rank_[rootX] > rank_[rootY]) {  
        parent[rootY] = rootX;  
    } else {  
        parent[rootY] = rootX;  
        rank_[rootX]++; // 秩相等时, 合并后根的秩+1  
    }  
}
```

7.4 图的最小代价生成树

稠密网（邻接矩阵）比较适合用Prim算法；稀疏网（邻接表）比较适合用Kruskal算法。

Prim算法的时间复杂度： $O(n^2)$

Kruskal算法的时间复杂度： $O(n + elbn + elbe)$

一般简化为 $O(elbe)$

为什么 “稠密网 + Prim + 邻接矩阵” “稀疏网 + Kruskal + 邻接表” 是最佳组合？

- 稠密图的核心特点是「边多、 $n^2 \approx E$ 」，而 Prim 算法的瓶颈恰好能被邻接矩阵的优势抵消：Prim 的核心操作需要 “快速查询边权”：Prim 每次要找 “当前生成树到未选顶点的最小边”，本质是 “查询已选顶点集与每个未选顶点的最小距离”。用邻接矩阵时，查询任意两个顶点的边权是 $O(1)$ ，遍历一个顶点的所有邻边（找最小边）只需扫一行 ($O(n)$)；整个算法的时间复杂度是 $O(n^2)$ (n 个顶点，每个顶点遍历一行)，而稠密图的 $E \approx n^2$ ， $O(n^2)$ 恰好等于 $O(E)$ ，效率极高。
- 邻接矩阵存储稠密图无浪费，若用邻接表存储稠密图，每个顶点的邻边链表都很长（接近 n 个节点），遍历邻边的成本和邻接矩阵差不多，但查询边权的效率 ($O(n)$) 远低于邻接矩阵 ($O(1)$)，反而更麻烦。
- Kruskal 在稠密图中效率极低：Kruskal 的瓶颈是 “边排序”，稠密图的 $E \approx n^2$ ，排序的时间复杂度是 $O(E \log E) \approx O(n^2 \log n^2) = O(n^2 \log n)$ ，比 Prim 的 $O(n^2)$ 慢一个 $\log n$ 量级，完全不划算。

为什么 “稠密网 + Prim + 邻接矩阵” “稀疏网 + Kruskal + 邻接表” 是最优组合？

- 稀疏网的核心特点是「边少、 E 远小于 n^2 」，而 Kruskal 的瓶颈能被邻接表的优势放大：Kruskal 的核心操作需要 “快速遍历所有边并排序”：Kruskal 的第一步是 “排序所有边”，稀疏网的 E 远小于 n^2 ，排序的时间复杂度 $O(E \log E)$ 会非常小。
- 用邻接表存储时，遍历所有边只需 $O(E)$ （直接遍历每个顶点的邻边链表），无需像邻接矩阵那样扫整个 $n \times n$ 矩阵（ $O(n^2)$ ，大部分是无效边）；整个算法的时间复杂度 $O(E \log E)$ ，在稀疏网中远低于 Prim 的 $O(n^2)$ 。
- Prim 在稀疏网中优化收益有限：Prim（邻接矩阵）的复杂度是 $O(n^2)$ ，在稀疏网中 n^2 远大于 E ，效率极低；即使优化为 “优先队列 + 邻接表”，复杂度也只能降到 $O(E \log n)$ ，和 Kruskal 的 $O(E \log E)$ 接近，但 Kruskal 的逻辑更简单（无需维护 “当前最小边集”），且排序 + 并查集的实现成本更低，实际应用中更高效。

7.5 有向无环图及其应用

- 一个无环的有向图称做有向无环图，简称DAG图。DAG图是一类较有向树更一般的特殊有向图。
- 除了很小的工程外，一般都把工程分为若干个叫做“活动”的子工程。2个问题：1是工程能否顺利进行；2是估算整个工程完成所必须的最短时间

7.5.1 拓扑排序

- 工程能否顺利进行，实际上就是一个拓扑排序的问题。
- 用顶点表示活动，用弧表示活动间的优先关系的有向无环图，称为顶点表示活动的图（Activity On Vertex Graph），简称为AOV图。

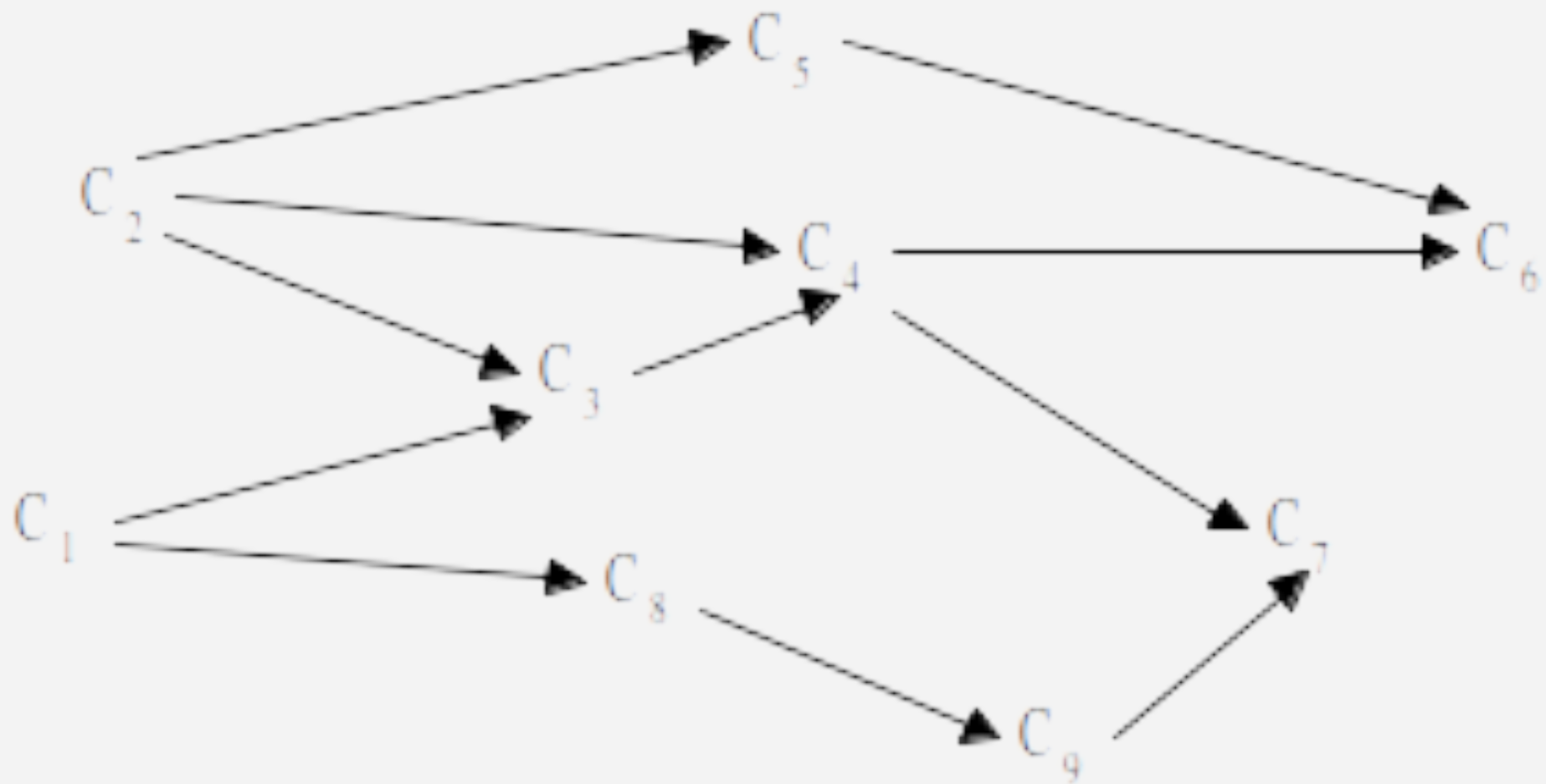
7.5.1 拓扑排序

- 在有向图 $G = (V, \{E\})$ 中, V 中顶点的线性序列 $(v_{i1}, v_{i2}, v_{i3}, \dots, v_{in})$ 称为拓扑序列, 序列必须满足如下条件: 对序列中任意两个顶点 v_i, v_j , 在 G 中有一条从 v_i 到 v_j 的路径, 则在序列中 v_i 必排在 v_j 之前。
- 下图的一个拓扑序列为: $C_1, C_2, C_3, C_4, C_5, C_8, C_9, C_7, C_6$ 。

7.5.1 拓扑排序

课程编号	课程名称	先修课程
C_1	高等数学	无
C_2	程序设计基础	无
C_3	离散数学	C_1, C_2
C_4	数据结构	C_2, C_3
C_5	算法语言	C_2
C_6	编译技术	C_4, C_5
C_7	操作系统	C_4, C_6
C_8	普通物理	C_1
C_9	计算机原理	C_8

7.5.1 拓扑排序



7.5.1 拓扑排序

AOV图的特性如下

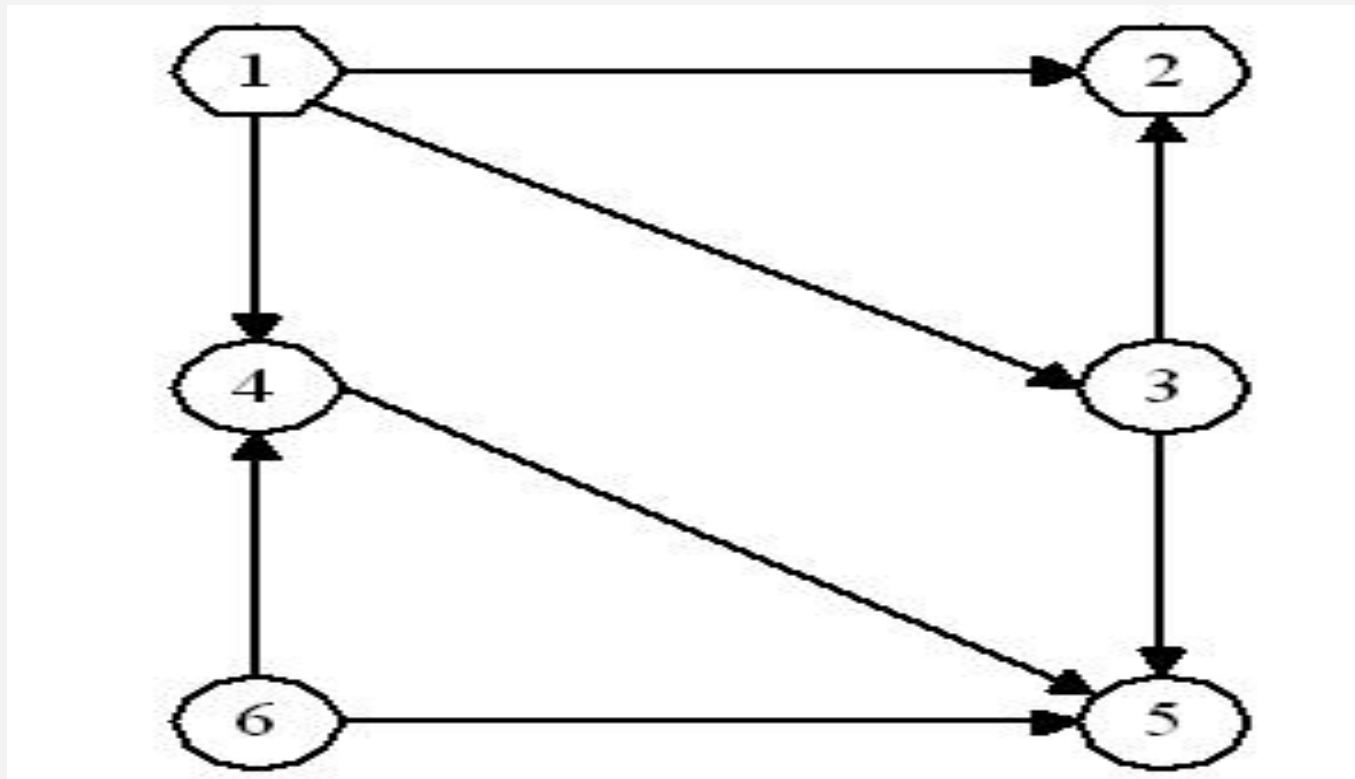
- 1、若 v_i 为 v_j 的先行活动， v_j 为 v_k 的先行活动，则 v_i 必为 v_k 的先行活动，即先行关系具有可传递性。从离散数学的观点来看，若有 $\langle v_i, v_j \rangle$ 、 $\langle v_j, v_k \rangle$ ，则必存在 $\langle v_i, v_k \rangle$ 。
- 2、在AOV图中不能存在回路，否则回路中的活动就会互为前驱，从而无法执行。
- 3、AOV图的拓扑序列不是唯一的。

7.5.1 拓扑排序

拓扑排序的基本方法

- (1) 从有向图中选一个无前驱的顶点输出；
- (2) 从有向图中删去此顶点以及所有以它为起点的边
- (3) 重复 (1)、(2)，直到不存在无前驱的顶点；
- (4) 若此时输出的顶点数小于有向图中的顶点数，则说明有向图中存在回路，否则输出的顶点的顺序即为一个拓扑序列。

7.5.1 拓扑排序



执行上述过程可以得到如下拓扑序列:

v1, v6, v4, v3, v2, v5 或 v1, v3, v2, v6, v4, v5

7.5.1 拓扑排序

- 没有前驱的顶点 °入度为零的顶点
- 删除顶点及以它为起点的边 °终点的入度减1
- 在算法中, 使用一个队列或栈存放入度为零的顶点, 供选择和输出无前驱的顶点。
- 算法的时间复杂度为 $O(n+e)$

7.5.1 拓扑排序

拓扑排序算法的描述

建立入度为零的顶点队列(或栈)

入度为零的队列(或栈)不空时, 重复执行

1. 从队列(或栈)中退出一个顶点, 并输出之
2. 从有向图中删去这个顶点和它发出的边, 边的终点入度减1
3. 如果边的终点入度减至0, 则该顶点进入入度为零的顶点队列(或栈)

如果输出顶点个数少于有向图的顶点个数, 则报告图中存在有向环

7.5.1 拓扑排序

```
/**
 * 函数：拓扑排序 (Kahn算法, 基于入度+队列)
 * 参数：G-有向图
 * 输出：拓扑序列 (若存在)，或提示图有环
 */
void TopologicalSort(ALGraph G) {
    int queue[MAX_VEX], front = 0, rear = 0; // 队列：存储入度为0的顶点
    int topoSeq[MAX_VEX], seqLen = 0;        // 拓扑序列及长度
    ArcNode *p;
    int u, v;

    // 步骤1：将所有入度为0的顶点入队
    for (u = 0; u < G.vexnum; u++) {
        if (G.vertices[u].inDegree == 0) {
            queue[rear++] = u;
        }
    }

    // 步骤2：处理队列中的顶点
    while (front < rear) {
        u = queue[front++]; // 出队 (无前置依赖的顶点)
        topoSeq[seqLen++] = u; // 加入拓扑序列
    }
}
```

7.5.1 拓扑排序

```
// 遍历u的所有邻接顶点v，删除u→v边（v的入度-1）
p = G.vertices[u].firstarc;
while (p != NULL) {
    v = p->adjvex;
    G.vertices[v].inDegree--; // 入度减1

    if (G.vertices[v].inDegree == 0) { // v无前置依赖，入队
        queue[rear++] = v;
    }

    p = p->nextarc;
}

// 步骤3：判断是否存在拓扑序列（序列长度是否等于顶点数）
if (seqLen == G.vexnum) {
    printf("拓扑排序成功! 序列为: ");
    for (int i = 0; i < seqLen; i++) {
        printf("%c ", G.vertices[topoSeq[i]].data);
    }
    printf("\n");
} else {
    printf("拓扑排序失败! 图中存在环，无合法拓扑序列。 \n");
}
}
```

7.5.2 关键路径

- 与AOV图相对应的是AOE网。即边表示活动的网。
- 用顶点表示事件，用边表示活动，边的权值表示活动所需要的时间。用这种方法构造的有向无环图叫做边表示活动的网（Activity On Edge Network），简称AOE网。

7.5.2 关键路径

AOE网在工程计划和管理中很有用。在研究实际问题时，人们通常关心的是

- 哪些活动是影响工程进度的关键活动？
- 至少需要多长时间能完成整个工程？

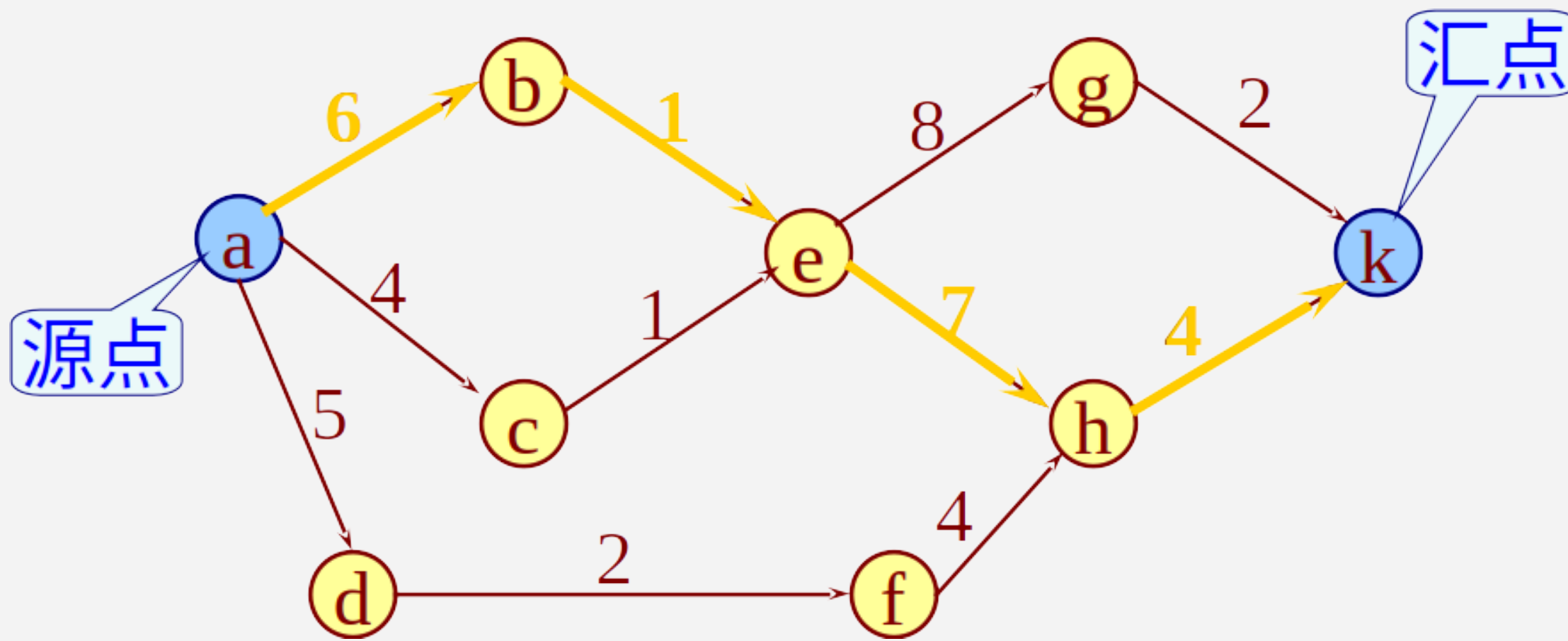
7.5.2 关键路径

- 在AOE网中存在唯一的、入度为零的顶点，叫做源点；存在唯一的、出度为零的顶点，叫做汇点。
- 从源点到汇点的最长路径的长度，即在这条路径上所有活动的持续时间之和，为完成整个工程任务所需的时间，称为最短完成时间；该路径叫做关键路径。

7.5.2 关键路径

- 关键路径上的活动叫做关键活动。
- 这些活动中的任意一项活动未能按期完成，则整个工程的完成时间就要推迟。相反，如果能够加快关键活动的进度，则整个工程可以提前完成。

7.5.2 关键路径



7.5.2 关键路径

- 事件 v_i 的最早发生时间 $ve(i)$ ：从源点到顶点 v_i 的最长路径的长度，叫做事件 v_i 的最早发生时间。
- 求 $ve(i)$ 时可从源点开始，按拓扑顺序向汇点递推
 - $ve(0)=0$
 - $ve(i)=\text{Max}\{ve(j) + g.\text{GetWeight}(j, i)\}; <j, i>\in T, 1\leq i\leq n-1;$
 - 其中， T 为所有以 i 为终点的边 $<j, i>$ 的集合， $g.\text{GetWeight}(j, i)$ 表示与边 $<j, i>$ 对应的活动的持续时间。

7.5.2 关键路径

- 事件 v_i 的最迟发生时间 $vl(i)$ ：是在不影响整个工程工期的情况下，事情的最迟发生时刻。在保证汇点按其最早发生时间发生这一前提下，求事件 v_i 的最迟发生时间。
- 在求出 $ve(i)$ 的基础上，可从汇点开始，按逆拓扑排序向源点递推，求出 $vl(i)$
 - $vl(n-1)=ve(n-1)$
 - $vl(i)=\text{Min}\{vl(j)-g.\text{GetWeight}(i, j)\} < i, j \in S, 0 \leq i \leq n-2;$
 - 其中， S 为所有以 i 为起点的边 $<i, j>$ 的集合， $g.\text{GetWeight}(i, j)$ 表示边与 $<i, j>$ 对应的活动的持续时间

7.5.2 关键路径

- 活动 $\langle v_i, v_j \rangle$ 的最早开始时间 $ee(i, j)$: 从源点到顶点 v_i 的最长路径的长度

$$ee(i, j) = ve(i)$$

- 活动 $\langle v_i, v_j \rangle$ 的最迟开始时间 $el(i, j)$: 是在不影响整个工程工期的情况下, 活动的最迟开始时间; 即在保证事件 v_j 的最迟发生时间为 $vl(j)$ 的前提下, 活动的最晚开始时间。

$$el(i, j) = vl(j) - g.GetWeight(i, j)$$

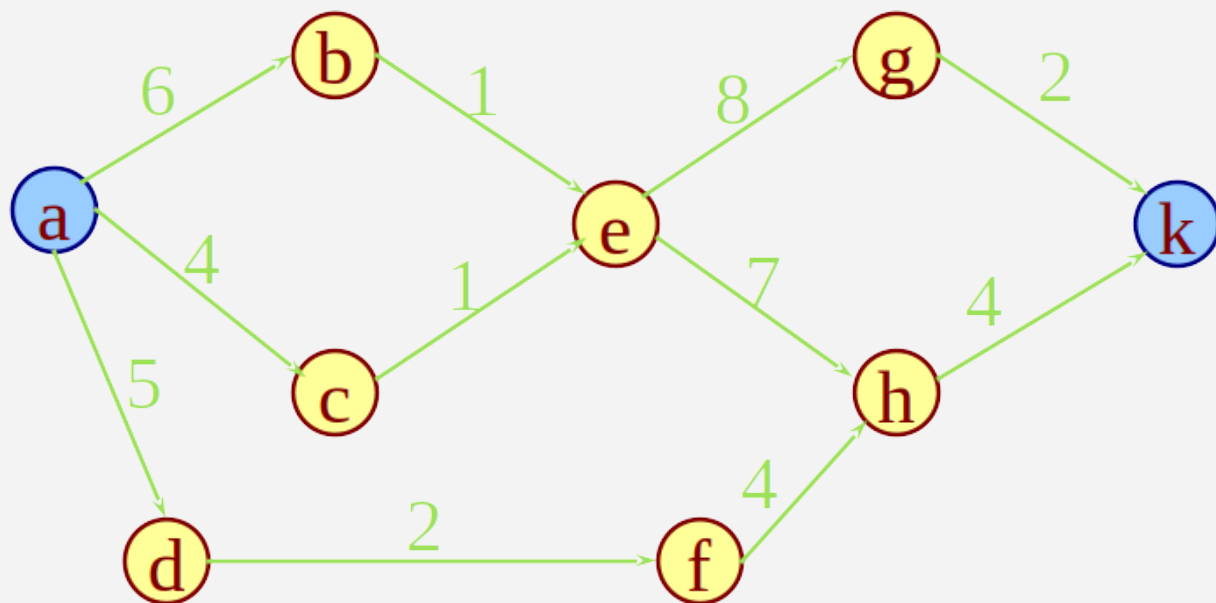
7.5.2 关键路径

- $el(i, j) - ee(i, j)$ 为活动 $\langle v_i, v_j \rangle$ 的时间余量。
- 将 $el(i, j) = ee(i, j)$ 的活动称为关键活动。
- 关键路径上的所有活动都是关键活动，只要找到了关键活动，就可以找到关键路径。

7.5.2 关键路径

求关键路径的基本步骤如下：

- (1) 对图中顶点进行拓扑排序，在排序过程中按拓扑序列求出每个事件的最早发生时间 $ve(i)$
- (2) 按逆拓扑序列求每个事件的最晚发生时间 $vl(i)$
- (3) 求出每个活动 $\langle v_i, v_j \rangle$ 的最早开始时间 $ee(i, j)$ 和最晚发生时间 $el(i, j)$
- (4) 找出 $ee(i, j) = el(i, j)$ 的活动 $\langle v_i, v_j \rangle$ 即为关键活动



拓扑有序序列： a - d - f - c - b - e - h - g - k

	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

	ab	ac	ad	be	ce	df	eg	eh	fh	gk	hk
权	6	4	5	1	1	2	8	7	4	2	4
ee	0	0	0	6	4	5	7	7	7	15	14
el	0	2	3	6	6	8	8	7	10	16	14
	☒			☒				☒			☒

7.5.2 关键路径

- 求ve的顺序应该是按拓扑有序序列的次序
- 求vl的顺序应该是按拓扑逆序序列的次序
- 拓扑逆序序列即为拓扑有序序列的逆序列
- 如果存在多条关键路径，单纯提高某一个子过程，并不能提高整个进度。
- 算法的时间复杂度为 $O(n+e)$

7.6 最短路径

- 假设要用一个网来表示交通运输网络，可以用顶点来表示城市，边表示城市之间的交通运输路线，边上的权值表示路线运输所花的时间。
- 本节讨论带权有向图，称路径上第一个顶点为源点，最后一个顶点为终点。路径长度度量路径上边的权值之和。

7.6 最短路径

- 最短路径问题: 如果有向网中某一顶点(称为源点)到达另一顶点(称为终点)的路径可能不止一条, 如何找到一条路径使得沿此路径上各边上的权值总和达到最小。

7.6 最短路径

最短路径问题解决：

- 单源点最短路径问题——迪杰斯特拉 (Dijkstra) 算法, 时间复杂度为 $O(n^2)$
- 所有顶点之间的最短路径——弗洛伊德 (Floyd) 算法, 时间复杂度为 $O(n^3)$

7.6.1 单源点最短路径问题

- 为求得这些最短路径，Dijkstra提出按路径长度的递增次序，逐步产生最短路径的算法。首先求出长度最短的一条最短路径，再参照它求出长度次短的一条最短路径，依次类推，直到从顶点 v_0 到其它各顶点的最短路径全部求出为止。
- 当我们按长度递增的顺序来产生各个最短路径时，设 U 为已经求得的最短路径的终点集合，可以证明：下一条最短路径或者是边 (v_0, v_x) ，或者是中间经过 U 中的某些顶点而后到达 v_x 的路径。

7.6.1 单源点最短路径问题

- 用反证法：假设下一条最短路径上有一个顶点 v_y 不在 U 中，即此路径为 $(v_0, \dots, v_y, \dots, v_x)$ 。显然， $(v_0, \dots, v_y)$ 的长度小于 $(v_0, \dots, v_y, \dots, v_x)$ 的长度，故下一条最短路径应为 $(v_0, \dots, v_y)$ ，这与假设的下一条最短路径 $(v_0, \dots, v_y, \dots, v_x)$ 相矛盾！因此，下一条最短路径上不可能有不在 U 中的顶点 v_y ，即假设不成立。

7.6.1 单源点最短路径问题

一般情况下，下一条长度最短的路径的长度必是： $\text{dist}[j] = \text{Min}\{\text{dist}[i] \mid v_i \in V - U\}$

其中， $\text{dist}[i]$ 或者是边 (v_0, v_i) 上的权值，或者是 $\text{dist}[k]$ ($v_k \in U$) 和边 (v_k, v_i) 上的权值之和。

U — 已求出的最短路径的终点集合 (开始为 $\{v_0\}$)。

$V-U$ — 尚未求出最短路径的顶点集合 (开始为 $V - \{v_0\}$ 的全部结点)

7.6.1 单源点最短路径问题

迪杰斯特拉算法的主要步骤

1、构造dist的初始值和路径path的初始值。dist[i]表示当前已求得的从源点 v_s 到 v_i 的最短路径长度；path[i]=-1表示还没有求出最短路径，path[i]=s表示路径上的上一顶点

$$dist[i] = \begin{cases} 0, & v_i = v_s \\ \infty, & \langle v_s, v_i \rangle \notin E \\ g.GetWeight, & \langle v_s, v_i \rangle \in E \end{cases}$$

$$path[i] = \begin{cases} s, & \langle v_s, v_i \rangle \in E \\ -1, & \langle v_s, v_i \rangle \notin E \end{cases}$$

7.6.1 单源点最短路径问题

迪杰斯特拉算法的主要步骤

2、选择 v_j 满足下面公式，将 v_j 加入到 U 中

$$dist[j] = \min_i \{ dist[j] \mid v_j \in V - U \}$$

7.6.1 单源点最短路径问题

迪杰斯特拉算法的主要步骤

3、修改从 v_s 到 $V-U$ 中任一顶点 v_k 的当前求得的最短路径，如果

$$dist[j] + g.GetWeight(j, k) < dist[k]$$

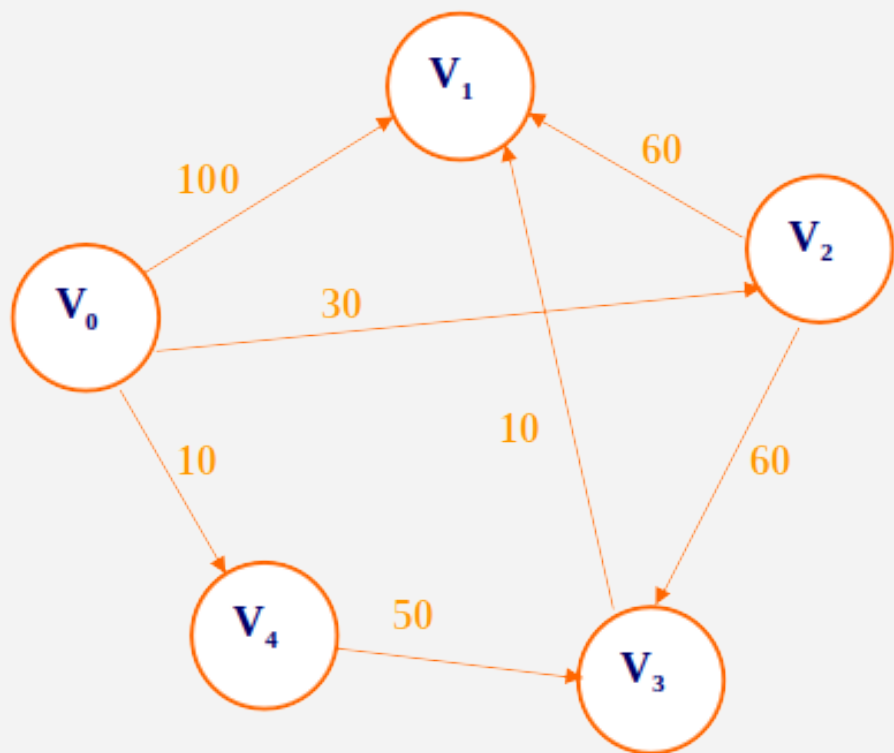
则新的从 v_s 到顶点 v_k 的当前求得的最短路径为

$(v_s, \dots, v_j, v_k)$ ，修改 $dist[k]$ 和 $path[k]$ 如下

$$dist[k] = dist[j] + g.GetWeight(j, k)$$

4、重复上述过程，求出所有从 v_s 到其余各点的最短路径

7.6.1 单源点最短路径问题



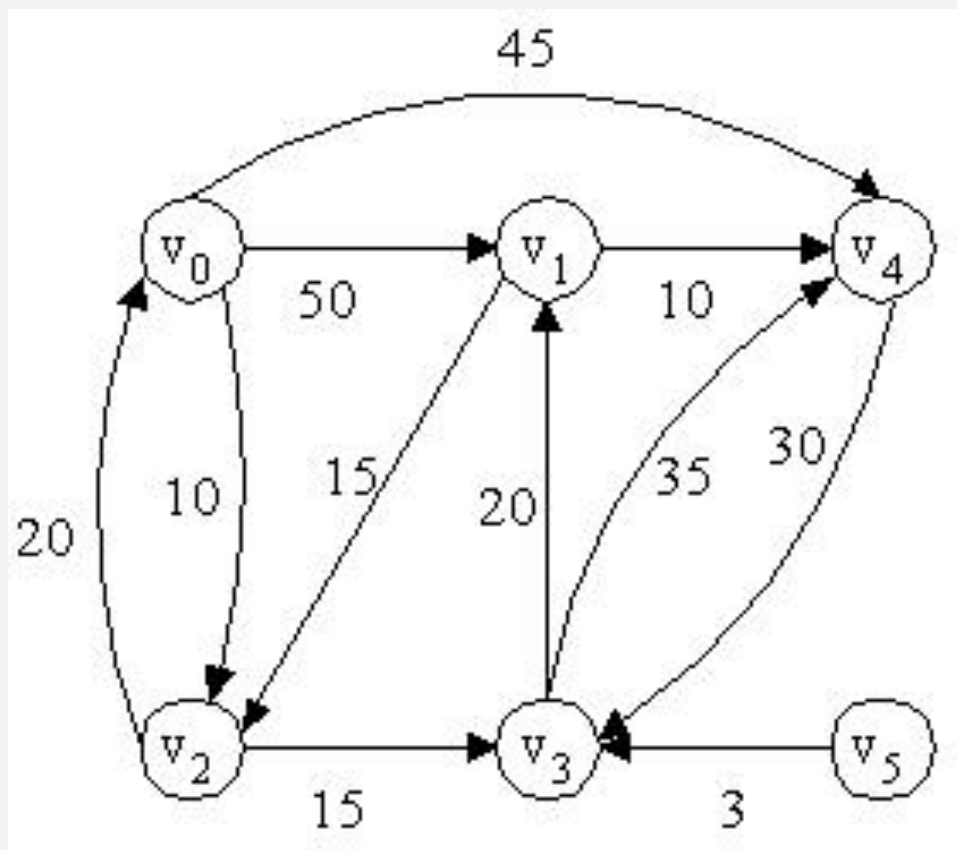
始点	终点	最短路径	路径长度
V_0	V_1	(V_0, V_4, V_3, V_1)	70
	V_2	(V_0, V_2)	30
	V_3	(V_0, V_4, V_3)	60
	V_4	(V_0, V_4)	10

按长度递增的次序，找到的路径顺序是
 V_4 , V_2 , V_3 , V_1

7.6.1 单源点最短路径问题

终点	i=1	i=2	i=3	i=4
V_1	dist[1]=100 path[1]=0	dist[1]=100 path[1]=0	dist[1]=90 path[1]=2	dist[1]=70 path[1]=3
V_2	dist[2]=30 path[2]=0	dist[2]=30 path[2]=0		
V_3	dist[3]= ∞ path[3]=-1	dist[3]=60 path[3]=4	dist[3]=60 path[3]=4	
V_4	dist[4]=10 path[4]=0			
V_j	V_4	V_2	V_3	V_1
U	{ v_0, v_4 }	{ v_0, v_2, v_4 }	{ v_0, v_2, v_3, v_4 }	{ v_0, v_1, v_2, v_3, v_4 }

7.6.1 单源点最短路径问题



		0	1	2	3	4	5
0		0	50	10	∞	45	∞
1		∞	0	15	∞	10	∞
2		20	∞	0	15	∞	∞
3		∞	20	∞	0	35	∞
4		∞	∞	∞	30	0	∞
5		∞	∞	∞	3	∞	0

7.6.1 单源点最短路径问题

源点	终点	最短路径	路径长度
V_0	V_2	V_0, V_2	10
	V_3	V_0, V_2, V_3	25
	V_1	V_0, V_2, V_3, V_1	45
	V_4	V_0, V_4	45
	V_5	V_0, V_5	无最短路径


```
// Dijkstra算法：邻接矩阵版
// 参数：graph-邻接矩阵，n-顶点数，start-起点（顶点编号从0开始）
void Dijkstra(int graph[][MAXVEX], int n, int start) {
    int dist[MAXVEX];    // dist[i]: 起点到顶点i的最短距离
    int visited[MAXVEX]; // visited[i]: 标记顶点i是否已访问（1=已访问，0=未访问）
    int i, j, u, min;

    // 第一步：初始化dist和visited数组
    for (i = 0; i < n; i++) {
        dist[i] = graph[start][i]; // 初始距离=起点到i的直接边权
        visited[i] = 0;            // 所有顶点初始未访问
    }
    dist[start] = 0; // 起点到自身距离为0
    visited[start] = 1; // 起点标记为已访问

    // 第二步：迭代n-1次（找到n-1条最短路径，覆盖所有顶点）
    for (i = 1; i < n; i++) {
        min = INFINITY;
        u = -1;

        // 子步骤1：找到当前未访问且距离最小的顶点u
        for (j = 0; j < n; j++) {
            if (!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }
    }
}
```

```
if (u == -1) break; // 若u未找到,说明剩余顶点不可达,提前退出
```

```
visited[u] = 1; // 标记u为已访问
```

```
// 子步骤2: 松弛操作: 更新所有未访问顶点的距离
```

```
for (j = 0; j < n; j++) {
```

```
    // 条件: v未访问 + u到v有边 (graph[u][j]≠无穷大) + 新路径更短
```

```
    if (!visited[j] && graph[u][j] != INFINITY && (dist[u] + graph[u][j] < dist[j])) {
```

```
        dist[j] = dist[u] + graph[u][j];
```

```
    }
```

```
}
```

```
}
```

```
// 第三步: 输出结果
```

```
printf("起点 %d 到各顶点的最短距离: \n", start);
```

```
for (i = 0; i < n; i++) {
```

```
    if (dist[i] == INFINITY) {
```

```
        printf("顶点 %d: 不可达\n", i);
```

```
    } else {
```

```
        printf("顶点 %d: %d\n", i, dist[i]);
```

```
    }
```

```
}
```

```
}
```

7.6.2 所有顶点之间的最短路径

- 求每一对顶点之间的最短路径的一个可行方法是以每个顶点为源点，重复执行迪杰斯特拉算法 n 次，时间复杂度为 $O(n^3)$
- 求每对顶点间的最短路径的另一个方法是弗洛伊德（Floyd）算法，虽然时间复杂度仍是 $O(n^3)$ ，但是思路更简单。

7.6.2 所有顶点之间的最短路径

- 弗洛伊德算法的基本思想是:从 v_i 到 v_j 的所有可能存在的路径中, 选出一条长度最短的路径
- 设其顶点集为 $V = \{v_0, v_1, \dots, v_{n-1}\}$, 算法的基本步骤如下。

7.6.2 所有顶点之间的最短路径

弗洛伊德算法

•开始: 若 $\langle v_i, v_j \rangle$ 存在, 则存在路径 $\{v_i, v_j\}$, 路径中不含其它顶点, 相当于中间点属于集合 $U = \{\}$

$$D^{-1}[i][j] = \begin{cases} 0, & i = j \\ \infty, & i \neq j, \langle v_i, v_j \rangle \notin E \\ g.GetWeight(i, j), & \langle v_i, v_j \rangle \in E \end{cases}$$

7.6.2 所有顶点之间的最短路径

弗洛伊德算法

- 第0步: 若 $\langle v_i, v_0 \rangle, \langle v_0, v_j \rangle$ 存在, 则存在路径 $\{v_i, v_0, v_j\}$, 路径中间点所含顶点序号不大于0, 相当于中间点属于集合 $U = \{v_0\}$

$$D^{(0)}[i][j] = \text{Min} \{D^{(-1)}[i][0] + D^{(-1)}[0][j], D^{(-1)}[i][j]\}$$

其中 $D^{(0)}[i][j]$ 表示从 v_i 到 v_j 中间点属于集合 $U = \{v_0\}$ 的最短路径的长度

7.6.2 所有顶点之间的最短路径

弗洛伊德算法

•第1步: 若存在路径 $\{v_i, \dots, v_1\}$ 与 $\{v_1, \dots, v_j\}$, 则存在路径 $\{v_i, \dots, v_1, \dots, v_j\}$, 路径中间点所含顶点序号不大于1, 相当于中间点属于集合 $U = \{v_0, v_1\}$

$$D^{(1)}[i][j] = \min \{D^{(0)}[i][1] + D^{(0)}[1][j], D^{(0)}[i][j]\}$$

其中 $D^{(1)}[i][j]$ 表示从 v_i 到 v_j 中间点属于集合 $U = \{v_0, v_1\}$ 的最短路径的长度

7.6.2 所有顶点之间的最短路径

弗洛伊德算法

•第k步: 若存在路径 $\{v_i, \dots, v_k\}$ 与 $\{v_k, \dots, v_j\}$, 则存在路径 $\{v_i, \dots, v_k, \dots, v_j\}$, 其中间点所含顶点序号不大于k, 相当于中间点属于集合 $U = \{v_0, v_1, \dots, v_k\}$

$$D^{(k)}[i][j] = \min \{D^{(k-1)}[i][k] + D^{(k-1)}[k][j], D^{(k-1)}[i][j]\}$$

其中 $D^{(k)}[i][j]$ 表示从 v_i 到 v_j 中间点属于集合 $U = \{v_0, v_1, \dots, v_k\}$ 的最短路径的长度

7.6.2 所有顶点之间的最短路径

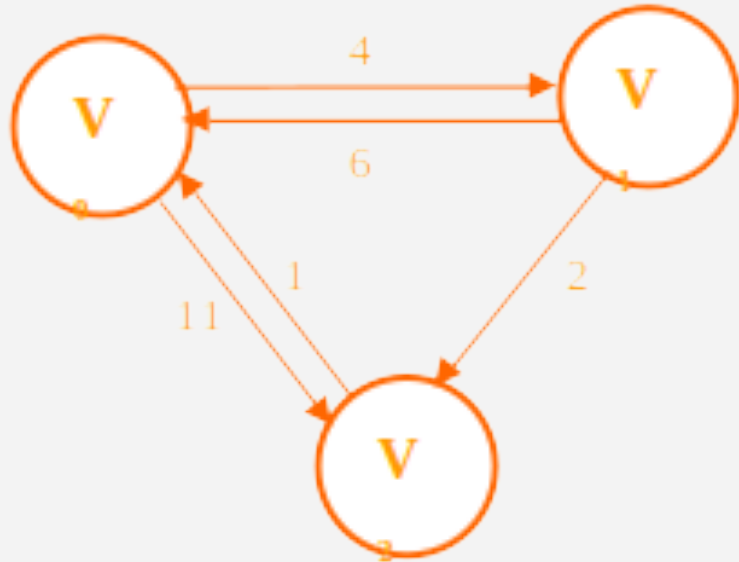
弗洛伊德算法

•第 $n-1$ 步: 若存在路径 $\{v_i, \dots, v_{n-1}\}$ 与 $\{v_{n-1}, \dots, v_j\}$, 则存在路径 $\{v_i, \dots, v_{n-1}, \dots, v_j\}$, 路径中间点所含顶点序号不大于 $n-1$, 相当于中间点属于集合 $U = \{v_0, v_1, \dots, v_{n-1}\}$

$$D^{(n-1)}[i][j] = \min \{D^{(n-2)}[i][n-1] + D^{(n-2)}[n-1][j], D^{(n-2)}[i][j]\}$$

其中 $D^{(n-1)}[i][j]$ 表示从 v_i 到 v_j 中间点属于集合 $U = \{v_0, v_1, \dots, v_{n-1}\}$ 的最短路径的长度

7.6.2 所有顶点之间的最短路径



$$dist^{(-1)} = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 1 & \infty & 0 \end{bmatrix}$$

$$path^{(-1)} = \begin{bmatrix} -1 & 0 & 0 \\ 1 & -1 & 1 \\ 2 & -1 & -1 \end{bmatrix}$$

$$dist^{(0)} = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 1 & 5 & 0 \end{bmatrix}$$

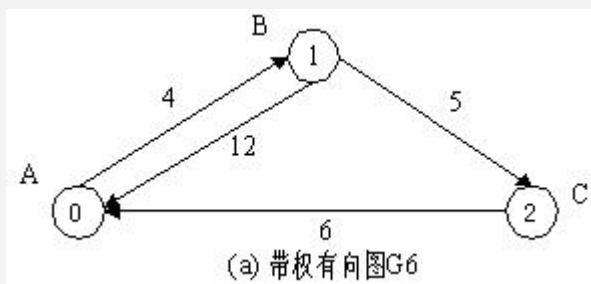
$$path^{(0)} = \begin{bmatrix} -1 & 0 & 0 \\ 1 & -1 & 1 \\ 2 & 0 & -1 \end{bmatrix}$$

$$dist^{(1)} = \begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 1 & 5 & 0 \end{bmatrix}$$

$$path^{(1)} = \begin{bmatrix} -1 & 0 & 1 \\ 1 & -1 & 1 \\ 2 & 0 & -1 \end{bmatrix}$$

$$dist^{(2)} = \begin{bmatrix} 0 & 4 & 6 \\ 3 & 0 & 2 \\ 1 & 5 & 0 \end{bmatrix}$$

$$path^{(2)} = \begin{bmatrix} -1 & 0 & 0 \\ 2 & -1 & 1 \\ 2 & 0 & -1 \end{bmatrix}$$



$$\begin{pmatrix} 0 & 4 & \infty \\ 12 & 0 & 5 \\ 6 & \infty & 0 \end{pmatrix}$$

D	D ⁽⁻¹⁾			D ⁽⁰⁾			D ⁽¹⁾			D ⁽²⁾		
	0	1	2	0	1	2	0	1	2	0	1	2
0	0	4	∞	0	4	∞	0	4	9	0	4	9
1	12	0	5	12	0	5	12	0	5	11	0	5
2	6	∞	0	6	10	0	6	10	0	6	10	0
P	P ⁽⁻¹⁾			P ⁽⁰⁾			P ⁽¹⁾			P ⁽²⁾		
	0	1	2	0	1	2	0	1	2	0	1	2
0		AB			AB			AB	ABC		AB	ABC
1	BA		BC	BA		BC	BA		BC	BCA		BC
2	CA			CA	CAB		CA	CAB		CA	CAB	

```

void Floyd(int graph[][MAXVEX], int n, int dist[][MAXVEX]) {
    int i, j, k;

    // 第一步：初始化dist矩阵（初始为直接边权）
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    // 第二步：迭代中间顶点k，松弛所有顶点对(i, j)
    for (k = 0; k < n; k++) { // k是中间顶点，必须放在最外层
        for (i = 0; i < n; i++) { // 起点i
            for (j = 0; j < n; j++) { // 终点j
                // 核心条件：
                // 1. i到k可达，k到j可达（避免INFINITY+INFINITY溢出）
                // 2. 经过k的路径（i→k→j）比当前路径（i→j）更短
                if (!IS_UNREACHABLE(dist[i][k]) && !IS_UNREACHABLE(dist[k][j])
                    && dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j]; // 更新最短距离
                }
            }
        }
    }
}

```

欲用四种颜色对地图上的国家涂色，有相邻边界的国家不能用同一种颜色（点相交不算相邻）。

(1) 试用一种数据结构表示地图上各国相邻的关系。

(2) 描述涂色过程的算法。（不要求证明）

地图涂色问题可以用“四染色”定理。将地图上的国家编号（1到n），从编号1开始逐一涂色，对每个区域用1色，2色，3色，4色（下称“色数”）依次试探，若当前所取颜色与周围已涂色区域不重色，则将该区域颜色进栈；否则，用下一颜色。若1至4色均与相邻某区域重色，则需退栈回溯，修改栈顶区域的颜色。用邻接矩阵数据结构 $C[n][n]$ 描叙地图上国家间的关系。n个国家用n阶方阵表示，若第i个国家与第j个国家相邻，则 $C_{ij}=1$ ，否则 $C_{ij}=0$ 。用栈s记录染色结果，栈的下标值为区域号，元素值是色数。

```
int n; // 实际国家数
int graph[MAX_COUNTRY][MAX_COUNTRY]; // 邻接矩阵 (存储相邻关系)
int color[MAX_COUNTRY]; // 存储每个国家的颜色 (1-4)

// 检查国家v使用颜色c是否安全 (与相邻国家颜色不冲突)
int isSafe(int v, int c) {
    for (int i = 0; i < n; i++) {
        // 若国家i与v相邻, 且i已染颜色c → 冲突
        if (graph[v][i] == 1 && color[i] == c) {
            return 0; // 不安全
        }
    }
    return 1; // 安全
}
```

```
int graphColoring(int v) {  
    // 终止条件：所有国家都染色完成  
    if (v == n) {  
        return 1;  
    }  
  
    // 尝试给当前国家v分配1-4种颜色  
    for (int c = 1; c <= MAX_COLOR; c++) {  
        if (isSafe(v, c)) { // 检查颜色c是否可用  
            color[v] = c;    // 分配颜色c  
  
            // 递归给下一个国家染色，若成功则返回1  
            if (graphColoring(v + 1)) {  
                return 1;  
            }  
  
            // 回溯：当前颜色c导致后续染色失败，撤销颜色  
            color[v] = 0;  
        }  
    }  
  
    // 4种颜色都尝试过，均冲突 → 回溯到上一层  
    return 0;  
}
```